

Міністерство освіти і науки України

Національний університет “Львівська політехніка”



Кафедра ЕОМ

ДОСЛІДЖЕННЯ ВУЗЛА ВВЕДЕННЯ ВІДЕОІНФОРМАЦІЇ В МПС

Методичні вказівки
до лабораторної роботи № 3
з дисципліни «Мікропроцесорні системи»
для студентів спеціальності 6.123.00.00 «Комп'ютерна інженерія»

Затверджено
на засіданні
кафедри електронних обчислювальних машин
Протокол №1 від 29.08.2019р.

Львів – 2019

Дослідження вузла введення відеоінформації в МПС: Методичні вказівки до лабораторної роботи № 3 з дисципліни «Мікропроцесорні системи» для студентів спеціальності 6.123.00.00 «Комп’ютерна інженерія» / Укладач: Пуйда В.Я. – Львів: Національний університет “Львівська політехніка”, 2019, 43 с.

Укладач

Пуйда В.Я., к. т. н, доцент

Рецензенти

Відповідальний за випуск:

Мельник А. О., професор, завідувач кафедри

ДОСЛІДЖЕННЯ ВУЗЛА ВВЕДЕННЯ ВІДЕОІНФОРМАЦІЇ В МПС

МЕТА РОБОТИ: ознайомлення з технічними засобами введення відеозображень в мікропроцесорні системи (МПС), архітектурою та особливістю функціонування цифрової відеокамери типу OV7670, інтерфейсом підключення цифрової відеокамери до МПС, з функціонуванням відеокамери в режимі введення відеозображень та виведенням на кольоровий дисплей в реальному часі. Ознайомитися з основними етапами та отримати навички розроблення драйвера вузла введення/виведення відеозображення в МПС.

ПОРЯДОК ВИКОНАННЯ РОБОТИ

1. Ознайомитися з лабораторним стендом на основі *на відлагоджувального модуля STM32F429 Discovery*.
2. Виконати у вказаному порядку основні етапи підготовки базової програми для введення/виведення відеозображення в МПС для з використанням STM32CubeMX (**ОСНОВНІ ЕТАПИ ВИКОНАННЯ РОБОТИ**).
3. Запрограмувати з допомогою утиліти **STM32 ST-LINK Utility** тестову програму **ov7670-i19341.hex** для введення відеозображень з цифрової відеокамери **OV7670** та виводу на кольоровий графічний індикатор **ILI9341**.
4. Використовуючи програмні модулі керування цифровою відеокамерою **OV7670** та графічним індикатором **ILI9341** згідно до завдання керівника підготувати програму для введення відеозображень з цифрової відеокамери **OV7670** та виводу на кольоровий графічний індикатор **ILI9341 відлагоджувального модуля STM32F429 Discovery**.
5. Відповісти на контрольні запитання.
6. Захистити лабораторну роботу.

ЗАВДАННЯ ДО ЛАБОРАТОРНОЇ РОБОТИ

1. Ознайомитися з методичними матеріалами до лабораторної роботи
2. Скачати STM32CubeMX з сайту
https://my.st.com/cas/login?service=https%3A%2F%2Fmy.st.com%2Fcontent%2Fmy_st_com%2Fen%2Fproducts%2Fdevelopment-tools%2Fsoftware-development-tools%2Fstm32-software-development-tools%2Fstm32-configurators-and-code-generators%2Fstm32cubemx.html
Для цього необхідно відкрити на вказаному сайті власний Account (**Create Account**)
3. Заінстальовати STM32CubeMX на власному ПК.
4. Ознайомитися з пакетом STM32CubeMX

ПИТАННЯ ДЛЯ САМОПЕРЕВІРКИ

1. Основні компоненти лабораторного стенда.
2. Основні параметри цифрової відеокамери **OV7670**.
3. Схема підключення цифрової відеокамери **OV7670** до відлагоджувального модуля **STM32F429 Discovery** лабораторного стенда.
4. Основні етапи підготовки програми в STM32CubeMX
5. Назвати засоби програмування пам'яті на кристалі мікропроцесорного компонента
6. Пояснити роботу підготовленої та відлагодженої програми.

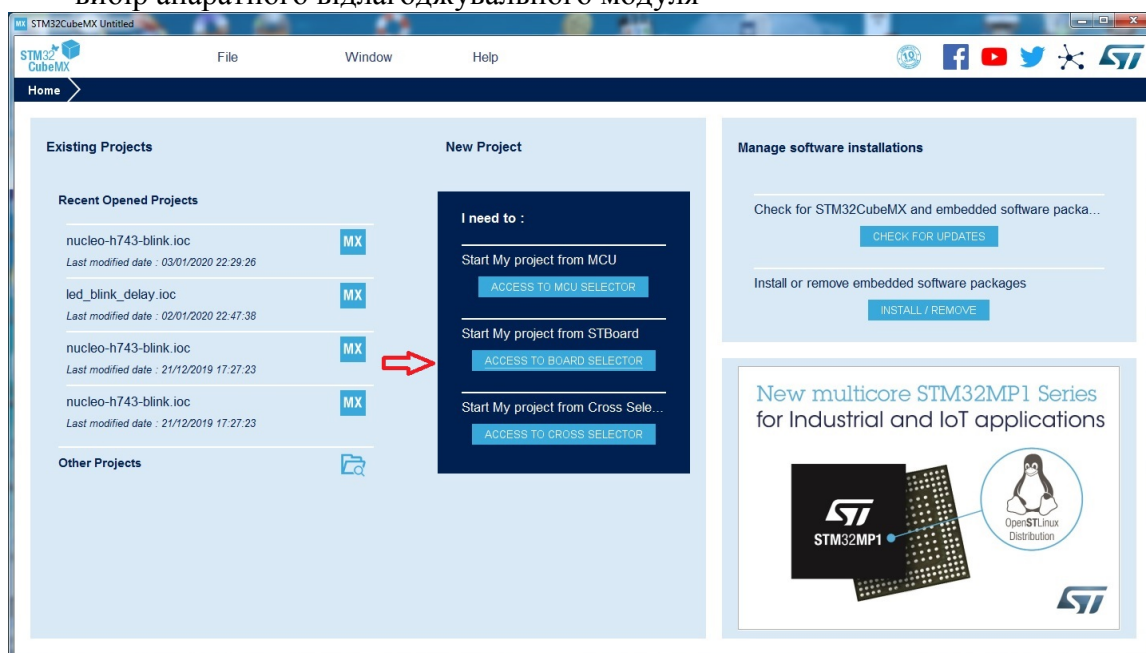
ОСНОВНІ ЕТАПИ ВИКОНАННЯ РОБОТИ

1. Запустити STM32CubeMX:



2. Поетапно виконати кроки по підготовці для IDE Keil 4 базового програмного коду для введення відеозображень з цифрової відеокамери **OV7670** та виводу на кольоровий графічний індикатор **ILI9341** відлагоджувального модуля **STM32F429 Discovery** :

- вибір апаратного відлагоджувального модуля



- вибір апаратного відлагоджувального модуля **STM32F429 Discovery**

New Project from a Board

MCU/MPU Selector Board Selector Cross Selector

Board Filters

Part Number Search

Vendor

Check/Uncheck All

☒ STMicroelectronics

Type

Check/Uncheck All

☒ Discovery kit

☐ Evaluation Board

☐ Evaluation Kit

☐ Nucleo USB Dongle

☐ Nucleo-RF Kit

☐ Nucleo144

☐ Nucleo32

☐ Nucleo64

MCU/MPU Series

Other

Price: From 0.0 to 99.0

Oscillator Freq: From 0 to 25 (MHz)

Peripheral

Features

Large Picture

Docs & Resources

Datasheet

Buy

Start Project

32F429IDISCOVERY

STMicroelectronics STM32F429I Discovery kit Board Support and Examples

Unit Price (US\$): 29.9

Mounted device: STM32F429ZITx

The 32F429IDISCOVERY kit leverages the capabilities of the STM32F429 high-performance microcontrollers, to allow users to easily develop rich applications with advanced Graphic User interfaces. With the latest board enhancement, the new STM32F429I-DISC1 order code has replaced the old STM32F429I-DISCO order code.

Features

- On-board ST-LINK/V2-1
- Supply through ST-Link USB
- External Supply : 3V and 5V
- JF3 (idd) for current measurement
- 54 Mbits SDRAM (1 Mbit x 16-bit x 4-bank)
- High-Speed USB OTG with micro-AB Connector
- TFT LCD 2.4" QVGA (240 x 320), 262K colors RGB
- Motion sensor, 3-axis digital output gyroscope (L3GD20)
- Two Push-buttons: User and Reset
- Six LEDs: USB COM, 3.3 V Power, user (Green/Red), USB-OTG (Green/Red)
- Extension header: (4 x 32) with 2.54 mm Pitch

Boards List: 40 items

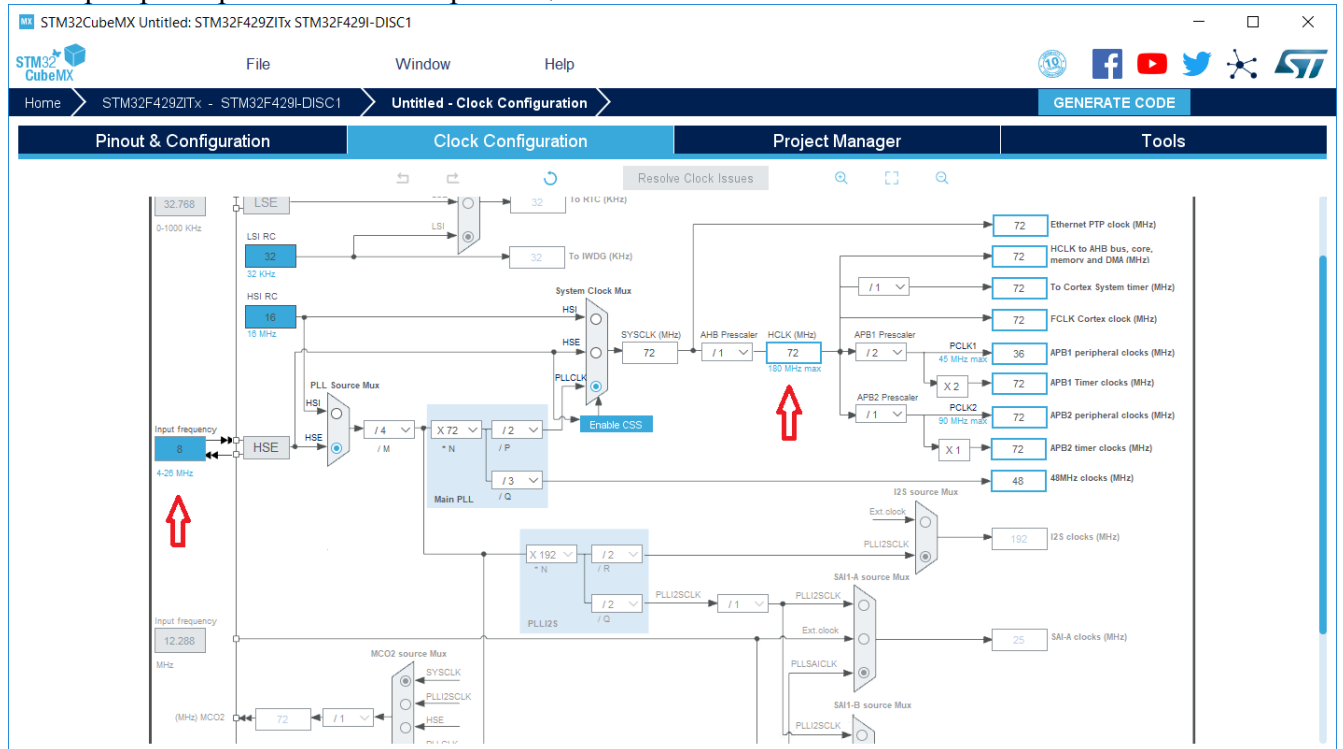
	Overview	Part No	Type	Marketing Status	Unit Price (US\$)	Mounted Device
		32F413HIDISCOVERY	Discovery kit	Active	70.0	STM32F413ZHTx
		32F429IDISCOVERY	Discovery kit	Active	29.9	STM32F429ZITx
		32F469IDISCOVERY	Discovery kit	Active	59.0	STM32F469NHTx

MX Board Project Options: STM32F429I-DI... X

? Initialize all peripherals with their default Mode ?

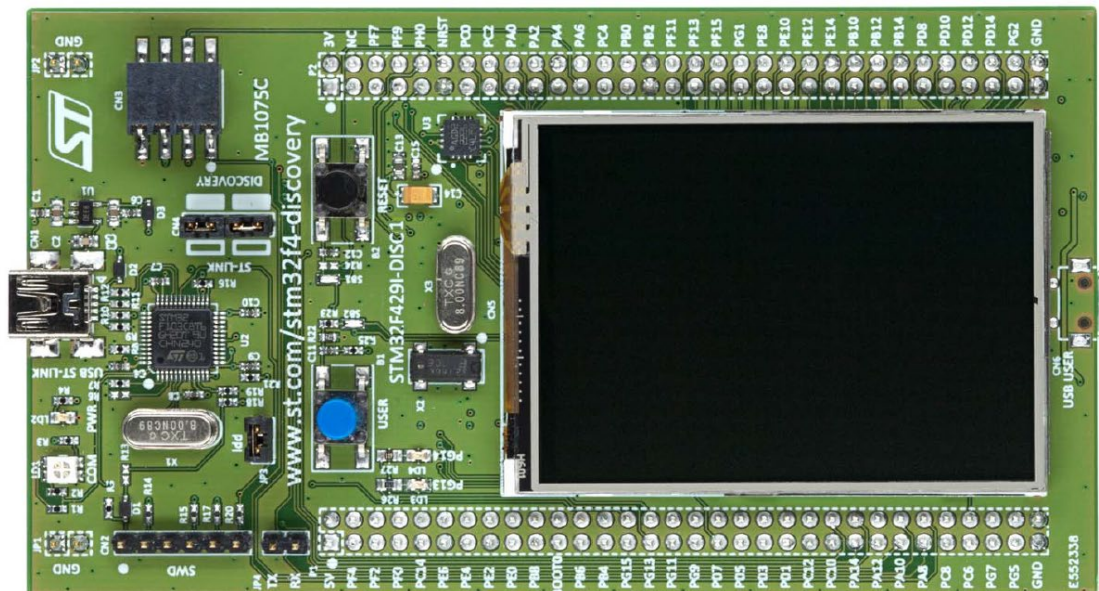
Yes No

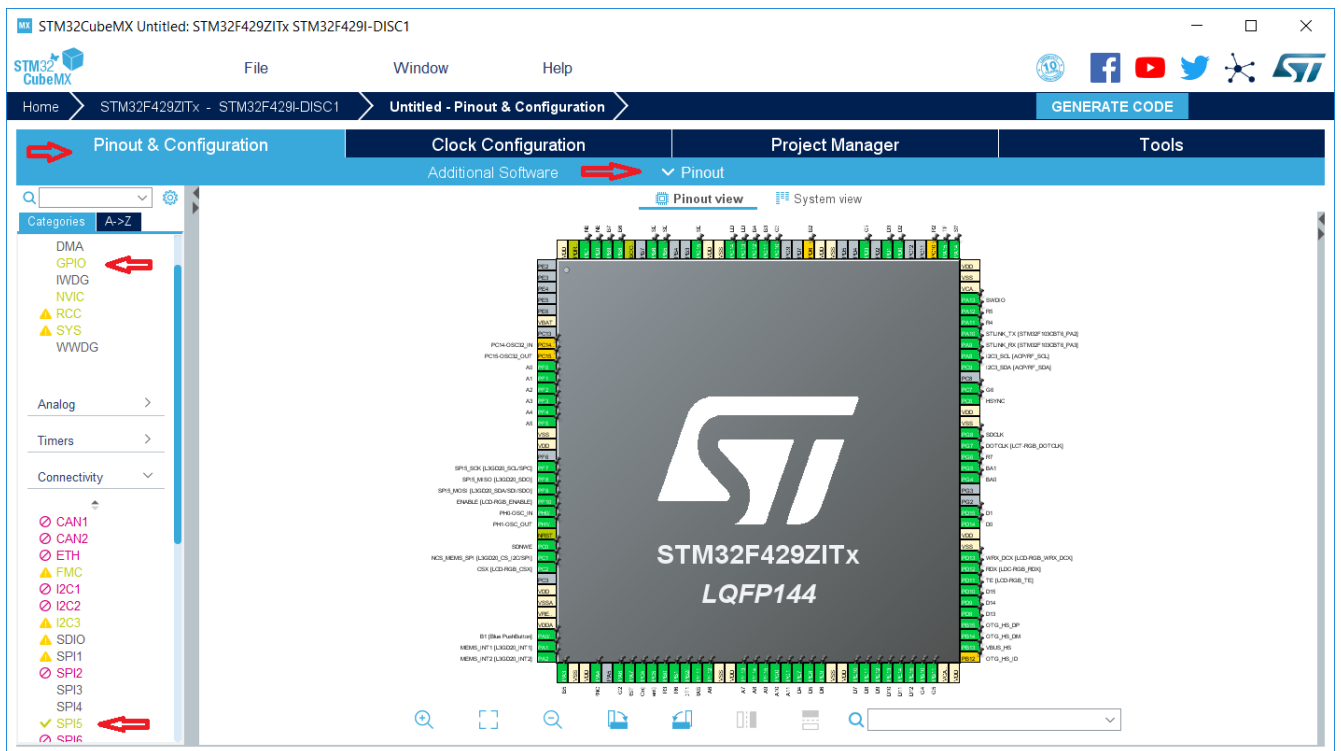
- вибір параметрів системи синхронізації



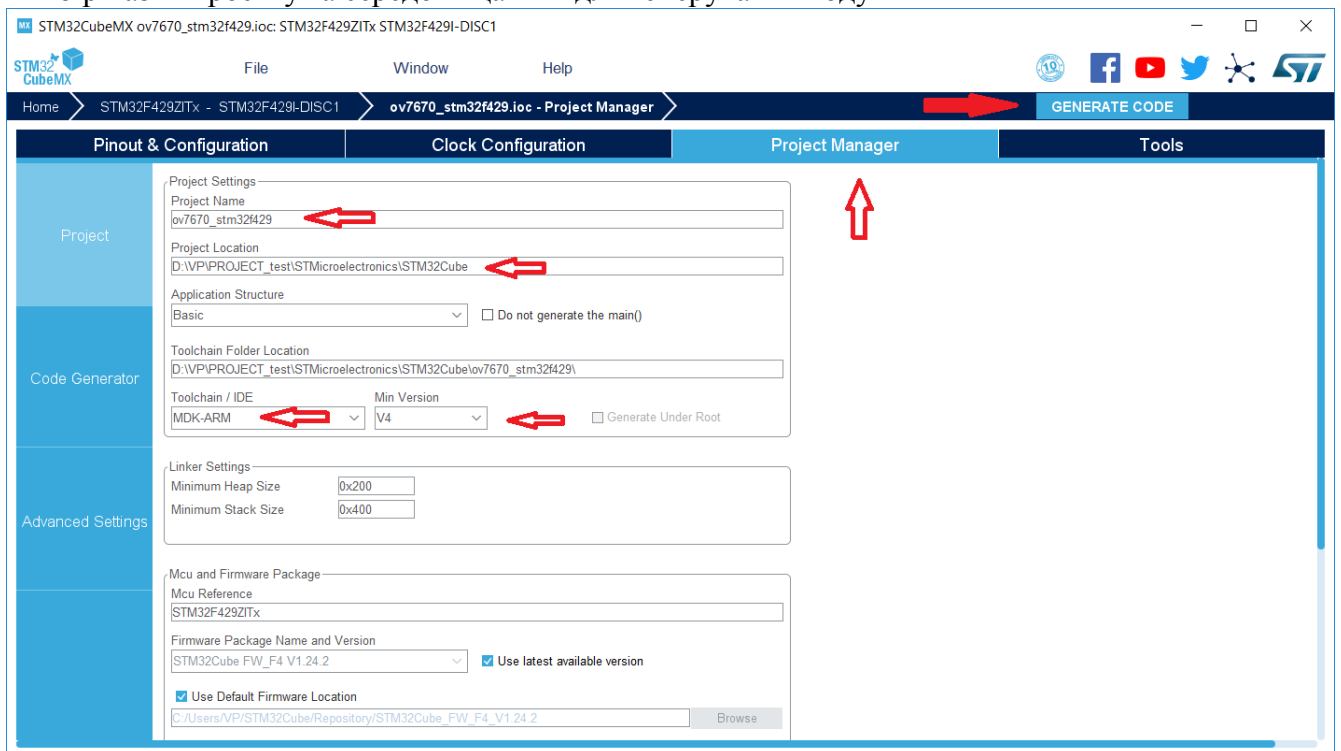
- вибір інтерфесу I2C та ліній I/O для керування і введення відеозображень з цифрової відеокамери **OV7670** та виводу на кольоровий графічний індикатор **ILI9341** відлагоджувального модуля **STM32F429 Discovery** та при необхідності ліній інших вузлів мікропроцесорної системи відповідного функціонального призначення

STM32F429 Discovery





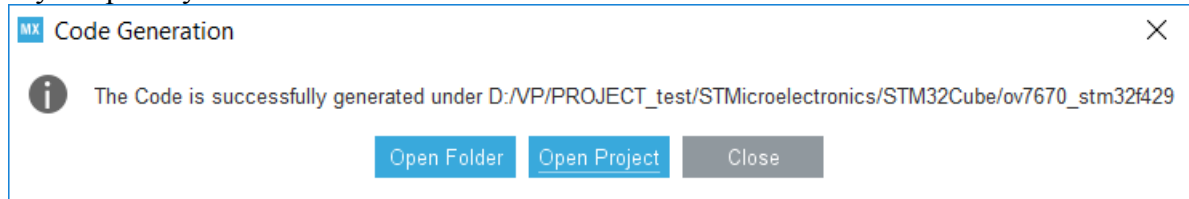
- вибір назви проекту та середовища IDE для генерування коду



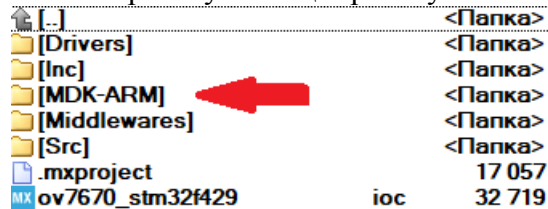
- процес генерування коду



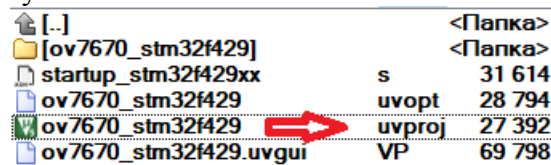
- запуск проекту



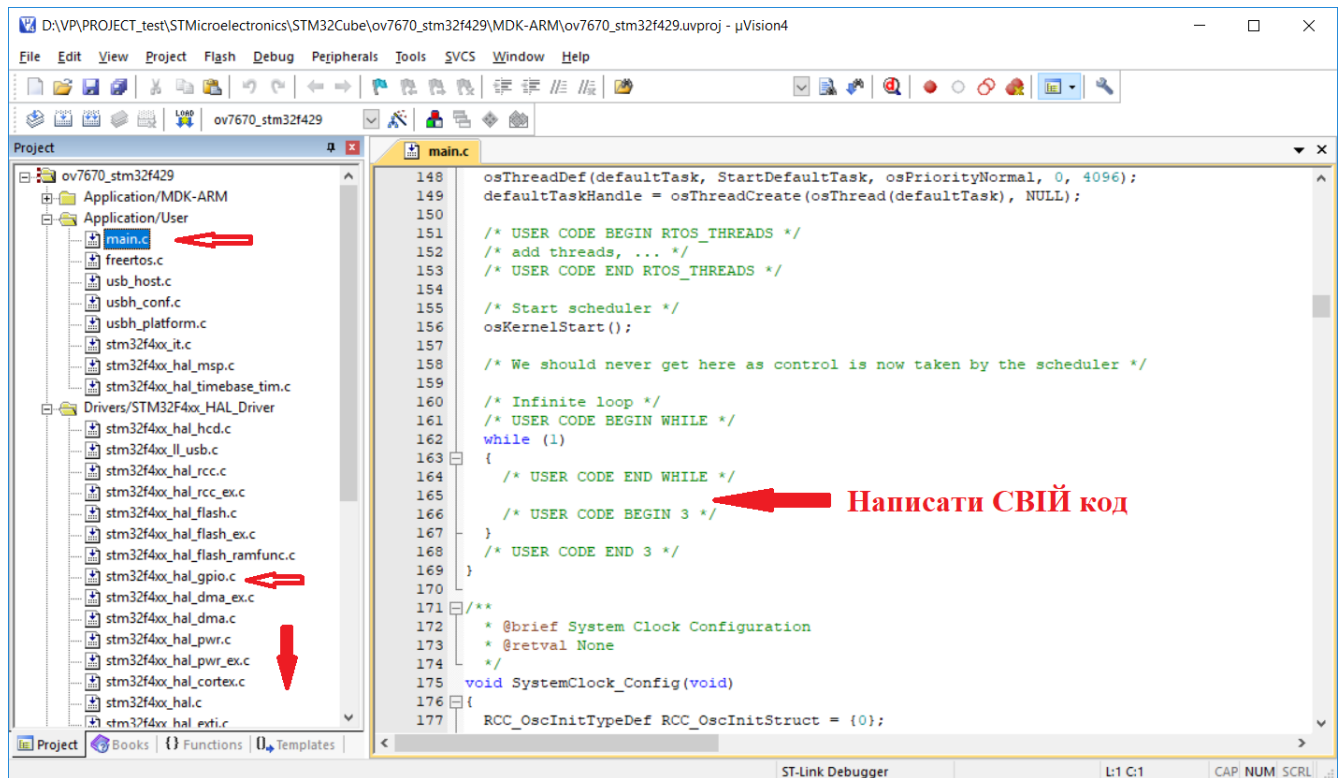
- розміщення згенерованого IDE проекту в папці проекту STM32CubeMX



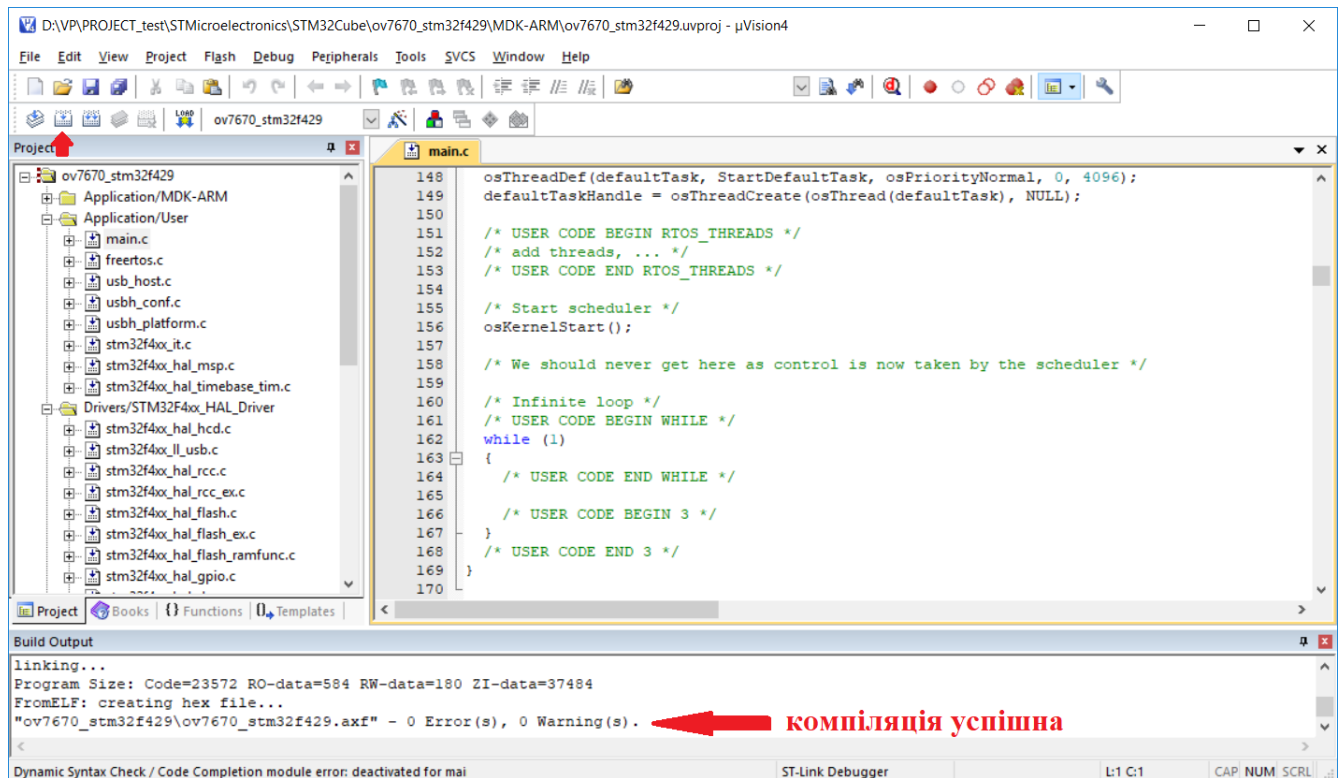
- файл запуску IDE проекту



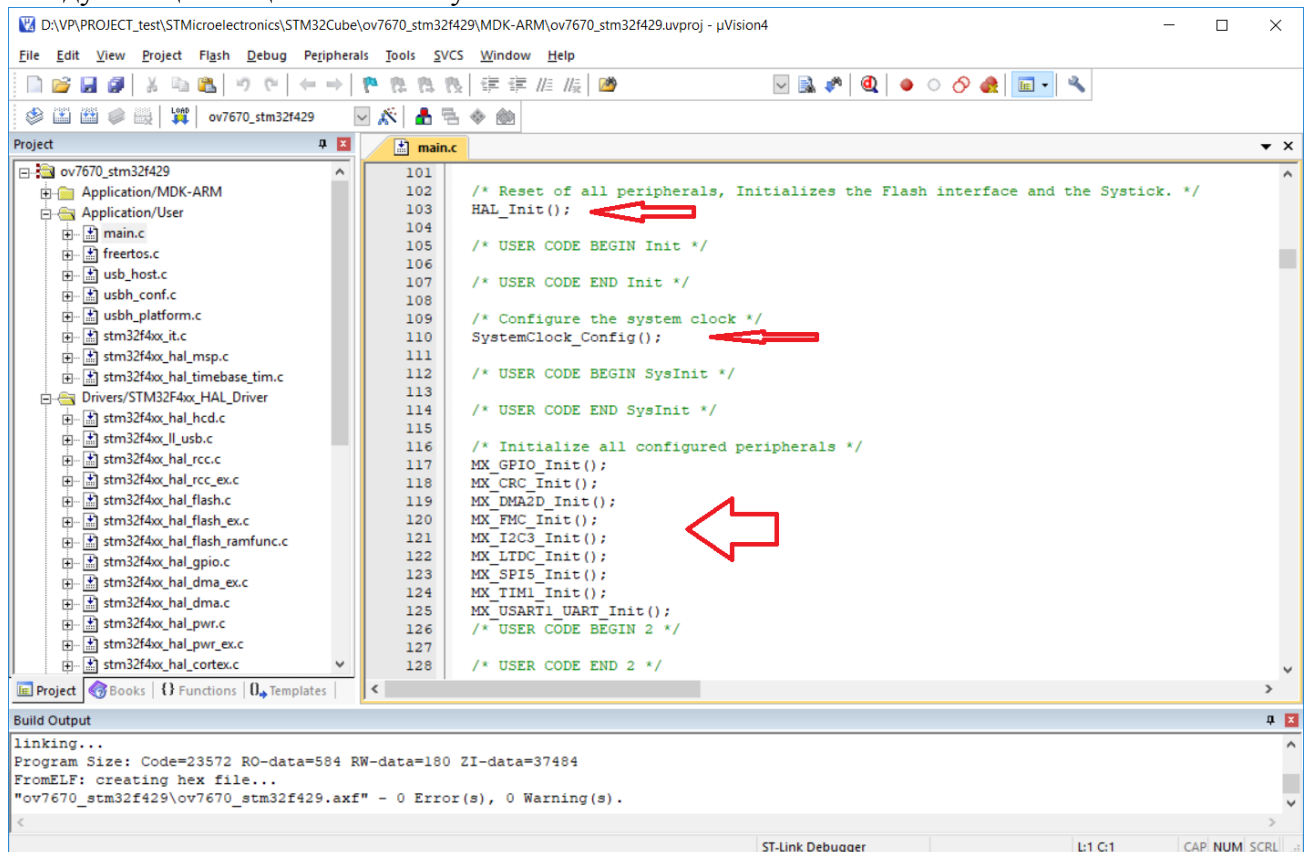
- вікно модуля main.c



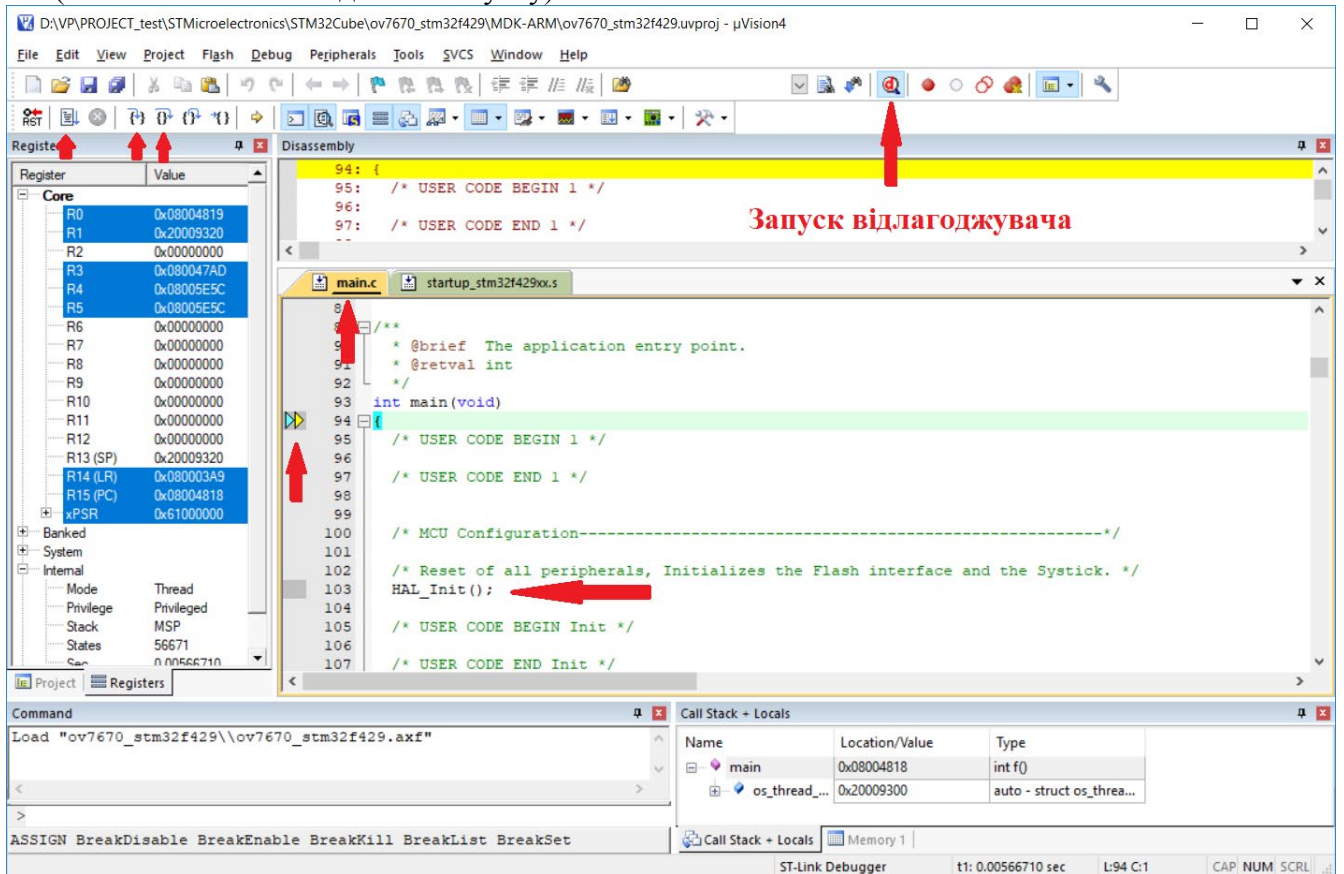
- запуск компіляції, виправлення помилок при їх наявності, перевірка та при необхідності корекція відповідних модулів ініціалізації SPI, I/O та інших вузлів у відповідності до мікропроцесорної системи конкретного функціонального призначення



- модулі ініціалізації основних вузлів



- підключити стенд до гнізда USB на ПК та запустити відлагоджувач
(вікно **main.c** після вдалого запуску)

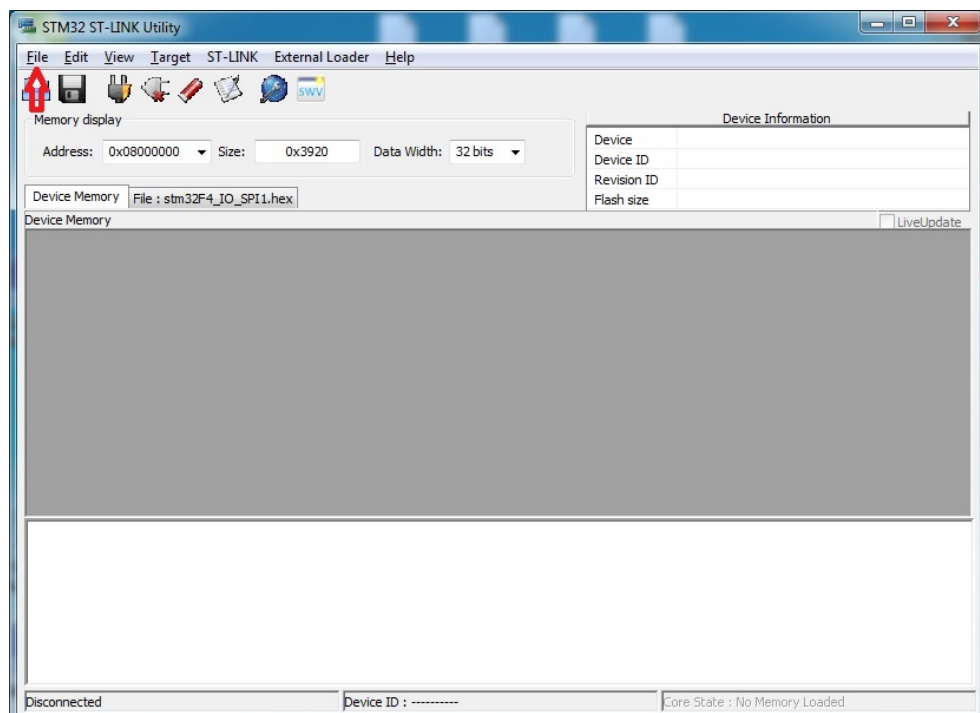
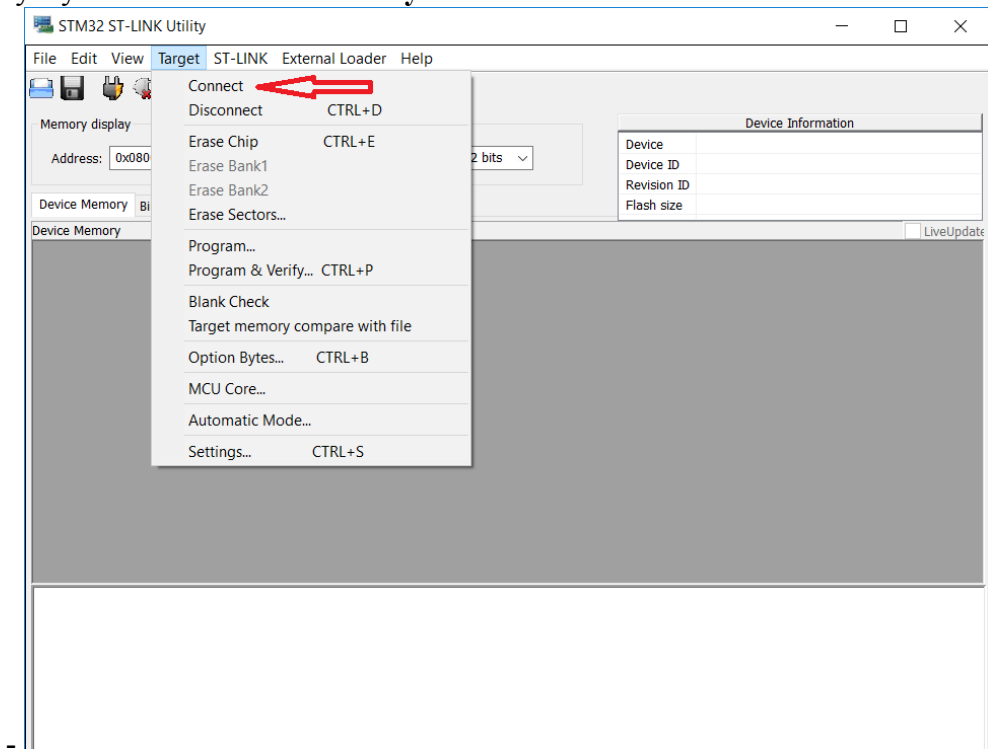


3. Запис завантажувального модуля програми в форматі *.hex в пам'ять на кристалі мікропроцесорного компонента

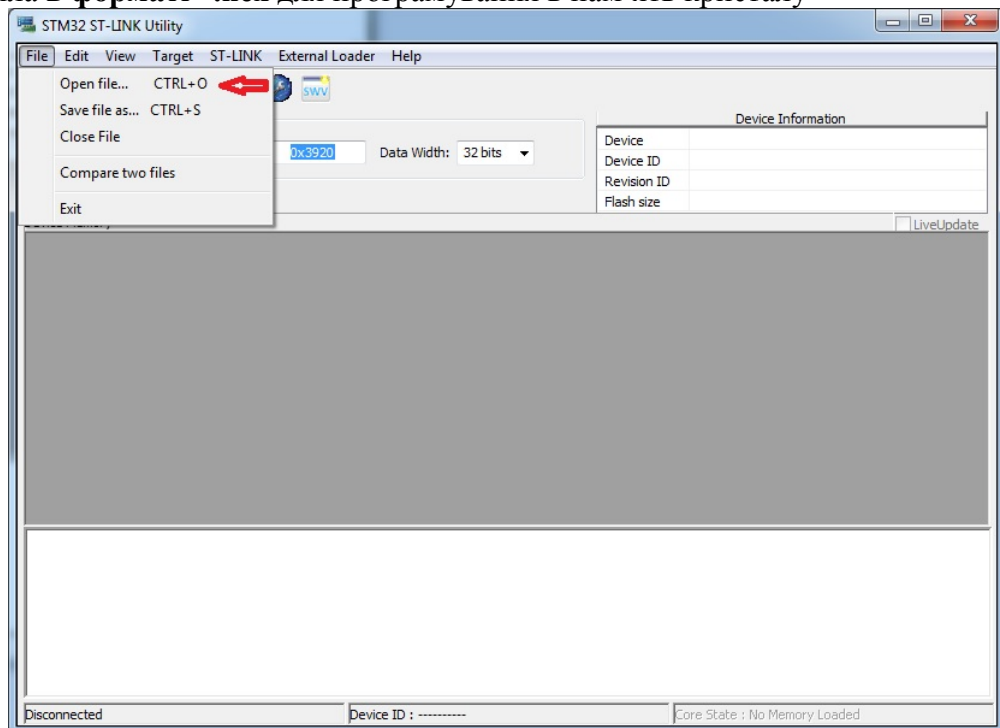
Програмування пам'яті на кристалі мікропроцесорного компонента можна здійснювати різними засобами:

- в процесі запуску відлагоджувача в режимі роботи з апаратурою;
- інтегрованим програматором в середовище IDE;
- окремою програмою, призначеною для роботи з відповідним мікропроцесорним сімейством,
наприклад, утилітою **STM32 ST-LINK Utility.exe**:

- вікно запуску **STM32 ST-LINK Utility**

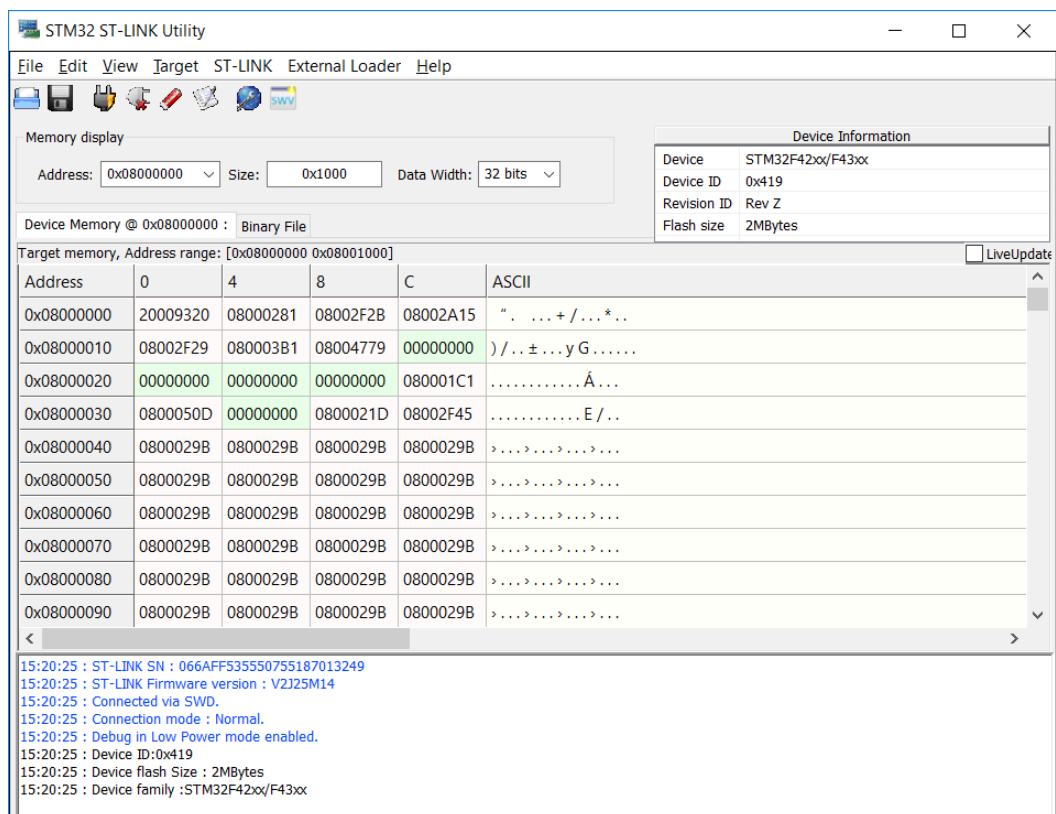


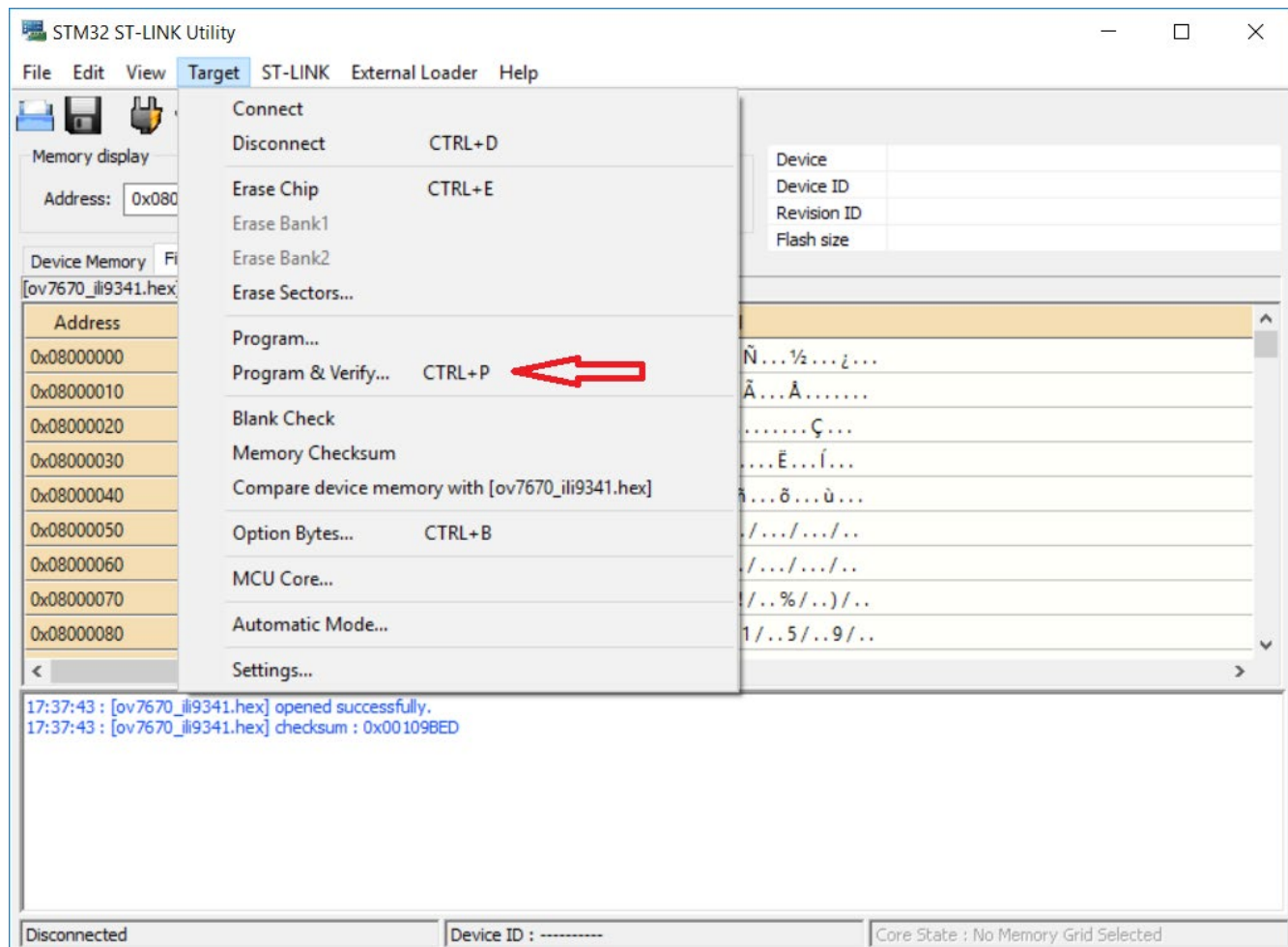
- вибір файла в форматі *.hex для програмування в пам'ять кристалу



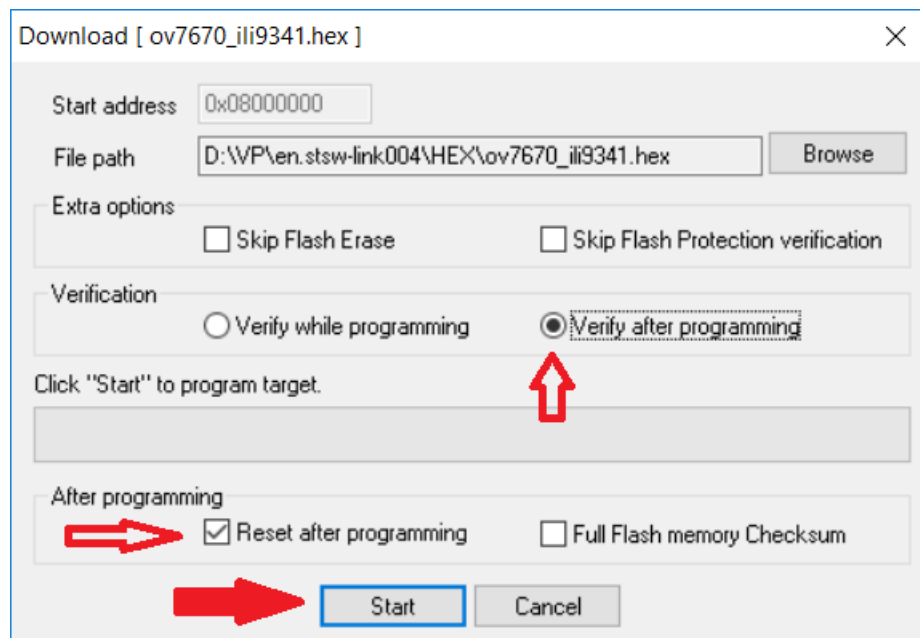
Ім'я

 ov7670_ili9341.hex





- вибір режимів програмування та запуск процесу програмування «Start»



- успішне завершення процесу програмування та верифікації запрограмованого коду з вхідним *.hex

The screenshot shows the STM32 ST-LINK Utility window. The 'Memory display' section is active, showing a table of memory addresses and their corresponding data. The 'Device' information is also visible on the right.

Memory display settings:
 Address: 0x08000000 | Size: 0x304C | Data Width: 32 bits

Device Information:
 Device: STM32F42xxx/F43xxx
 Device ID: 0x419
 Revision ID: Rev 3
 Flash size: 2MBytes

Target memory, Address range: [0x08000000 0x0800304C]

Address	0	4	8	C	ASCII
0x08000000	20025C38	08002ED1	08002EBD	08002EBF	8\ . Ñ...½...¿...
0x08000010	08002EC1	08002EC3	08002EC5	00000000	Á...Ã...À.....
0x08000020	00000000	00000000	00000000	08002EC7	Ç...
0x08000030	08002EC9	00000000	08002ECB	08002ECD	É.....Ê...Í...
0x08000040	08002EED	08002EF1	08002EF5	08002EF9	í...ñ...ô...ù...
0x08000050	08002EFD	08002F01	08002F05	08002F09	ý.../.../.../..
0x08000060	08002F0D	08002F11	08002F15	08002F19	./.../.../.../..
0x08000070	08002F1D	08002F21	08002F25	08002F29	./...!/...%/...)/..
0x08000080	08002F2D	08002F31	08002F35	08002F39	-/...1/...5/...9/..

Command Log:

```

17:39:10 : V23JUN20
17:39:18 : Connected via SWD.
17:39:18 : SWD Frequency = 4,0 MHz.
17:39:18 : Connection mode : Normal.
17:39:18 : Debug in Low Power mode enabled.
17:39:18 : Device ID:0x419
17:39:18 : Device flash Size : 2MBytes
17:39:18 : Device family :STM32F42xxx/F43xxx
17:42:15 : Memory programmed in 0s and 594ms.
17:42:15 : Verification...OK
  
```

Status Bar:
 Debug in Low Power mode enabled. | Device ID:0x419 | Core State : Live Update Disabled

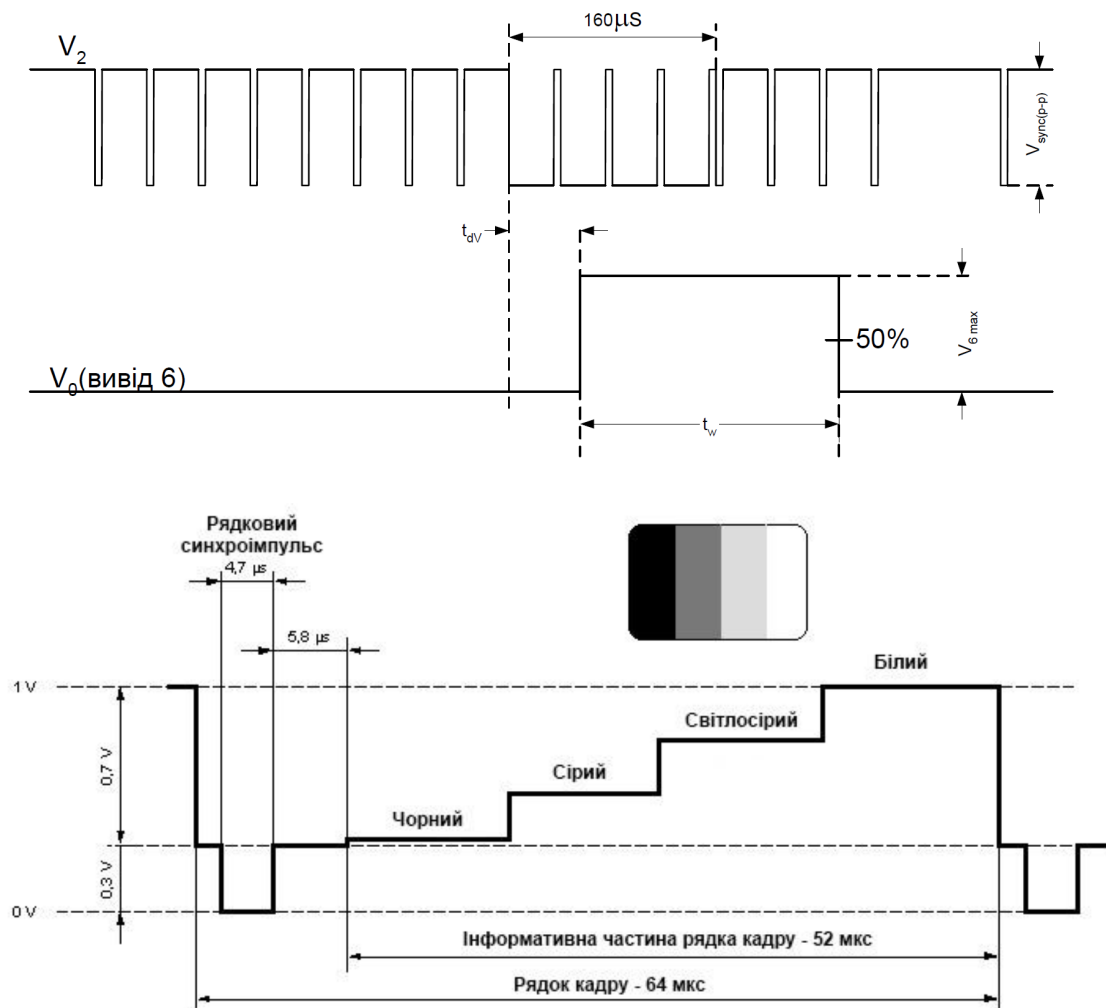
МЕТОДИЧНІ МАТЕРІАЛИ

Введення відеоінформації в мікропроцесорну систему (МПС) можна реалізувати на основі аналогової чи цифрової відеокамери. Існують різні типи відеокамер з різними технічними характеристиками та експлуатаційними параметрами.

Аналогові відеокамери

Прикладом застосування аналогової відеокамери є відеокамери огляду деяких автомобілів тощо.

Аналогова відеокамера формує аналоговий відеосигнал, який для введення в МПС необхідно перетворити в цифрову форму з використанням швидкодіючого аналого-цифрового перетворювача (АЦП). Відеосигнал монохромної (чорно-білої) аналогової відеокамери містить інформацію про яскравість елементів зображення, а також синхросигнали рядкових, кадрових та керуючих синхроімпульсів. Повний кольоровий телевізійний сигнал крім відеосигналу містить піднесучу, промодульовану сигналом кольору, що містить інформацію про колір елементів зображення, а також сигнал синхронізації кольору.



Відеосигнал монохромної відеокамери.

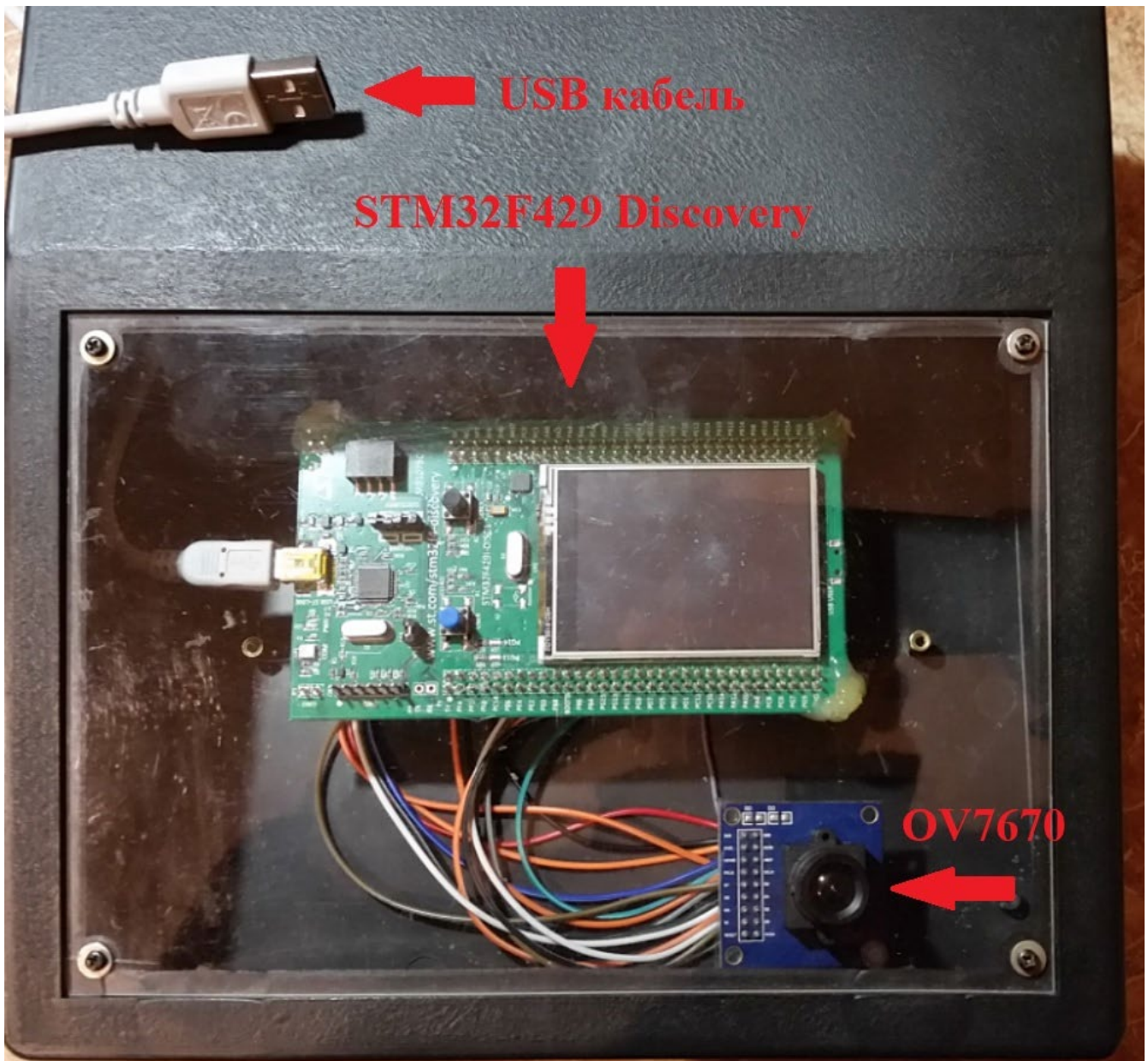
Цифрові відеокамери

Цифрові відеокамери формують на виході відеозображення в цифровому коді в різноманітних форматах. Передача відеоінформації в МПС здійснюється через паралельний чи швидкодіючий послідовний інтерфейс, наприклад, SPI. Для керування режимами функціонування цифрової відеокамери часто використовується інтерфейс I2C.

В лабораторній роботі досліджується введення відеоінформації в МПС з допомогою цифрової відеокамери OV7670, див. відповідний підрозділ методичних матеріалів.

1.Лабораторний стенд

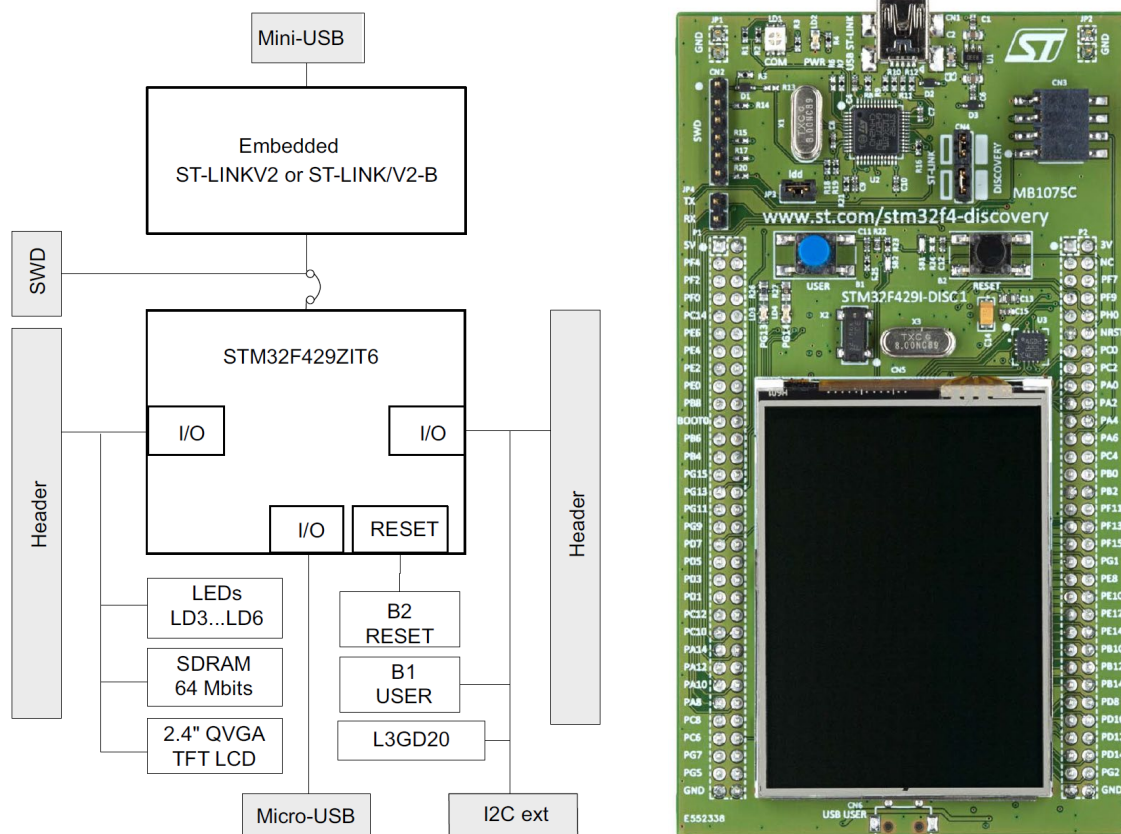
Стенд реалізовано на базі відлагоджувального модуля **STM32F429DISCOVERY** з підключеною цифровою відеокамерою **OV7670** та кольоровим графічним індикатором **ILI9341**. Підключення стенду здійснюється **USB** кабелем до любого порта **USB 2.0** чи **USB 3.0** комп'ютера.



Лабораторний стенд

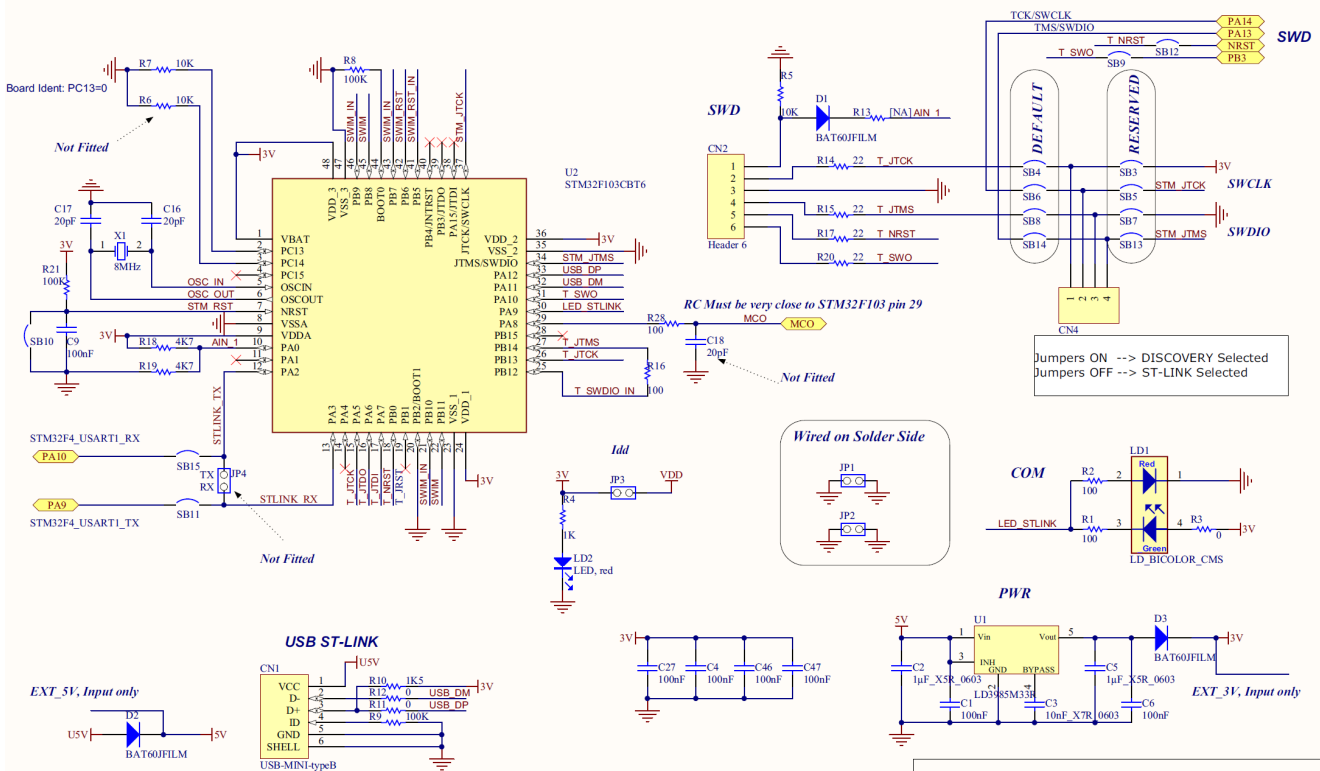
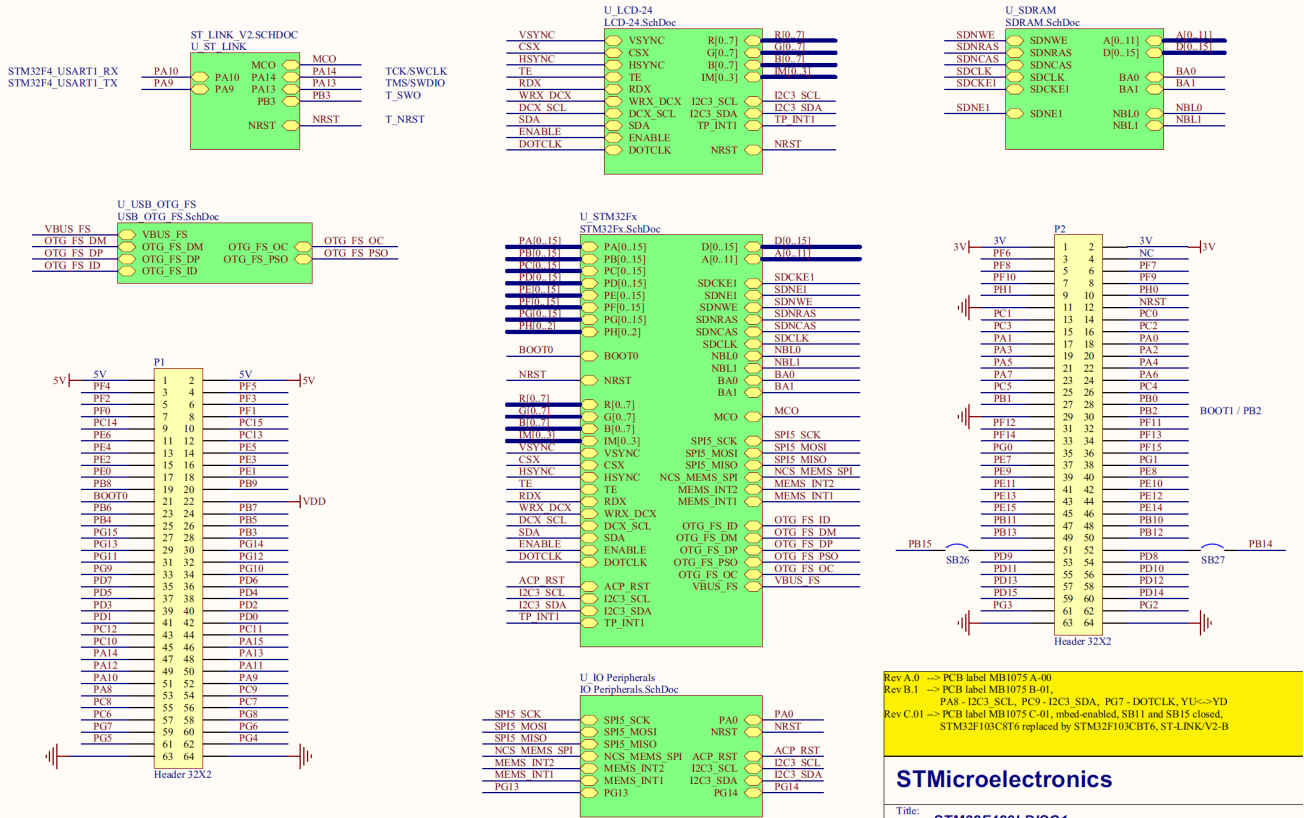
2. Відлагоджувальний модуль STM32F429DISCOVERY

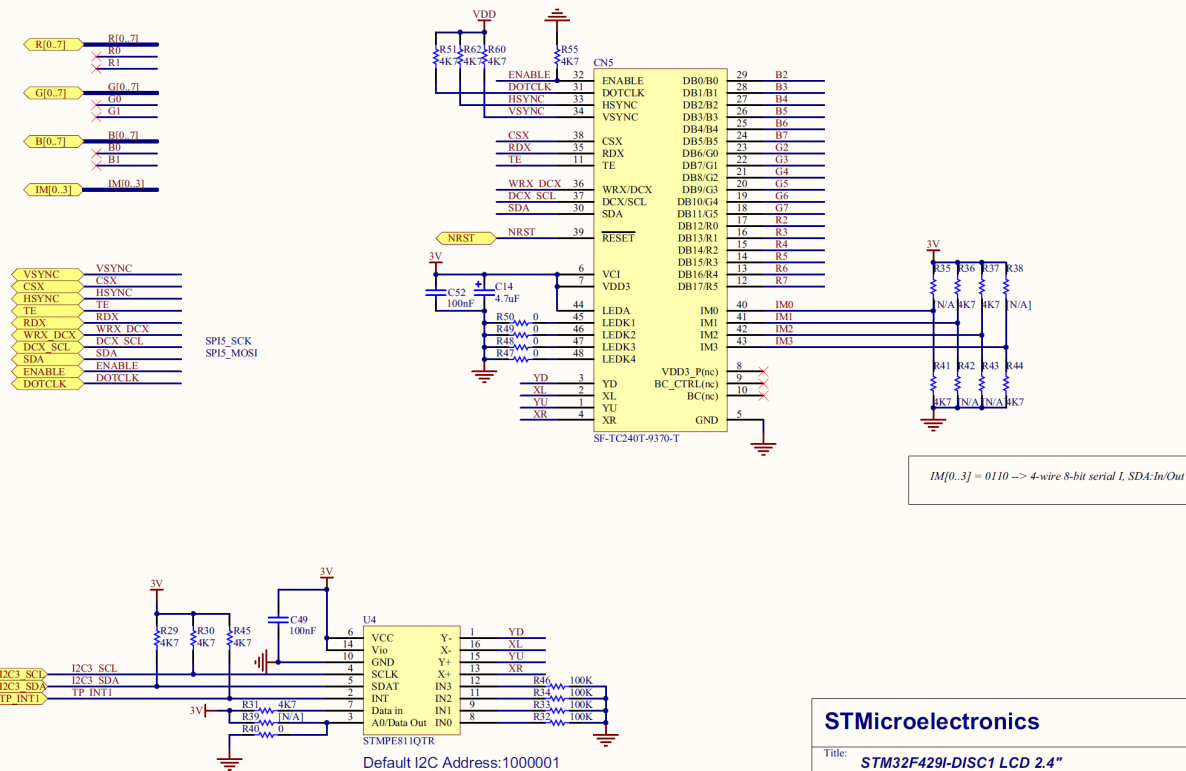
Відлагоджувальний модуль **STM32F429DISCOVERY** призначений для відлагодження в реальному часі програм для мікропроцесорних систем (МПС) різного функціонального призначення з метою розпаралелювання процесу розроблення апаратної частини МПС та програмного забезпечення до неї. **STM32F429DISCOVERY** реалізовано на базі мікроконтролера з ядром **Cortex-M4F** типу **STM32F429ZIT6** в корпусі **LQFP144**. Структурна схема та зовнішній вигляд **STM32F429DISCOVERY** зображені на рис.



Структурна схема та зовнішній вигляд **STM32F429DISCOVERY**

Cxema STM32F429DISCOVERY

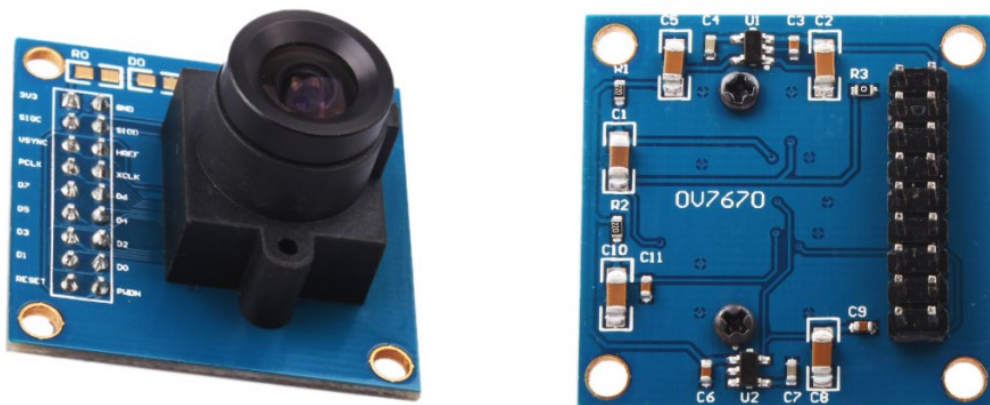




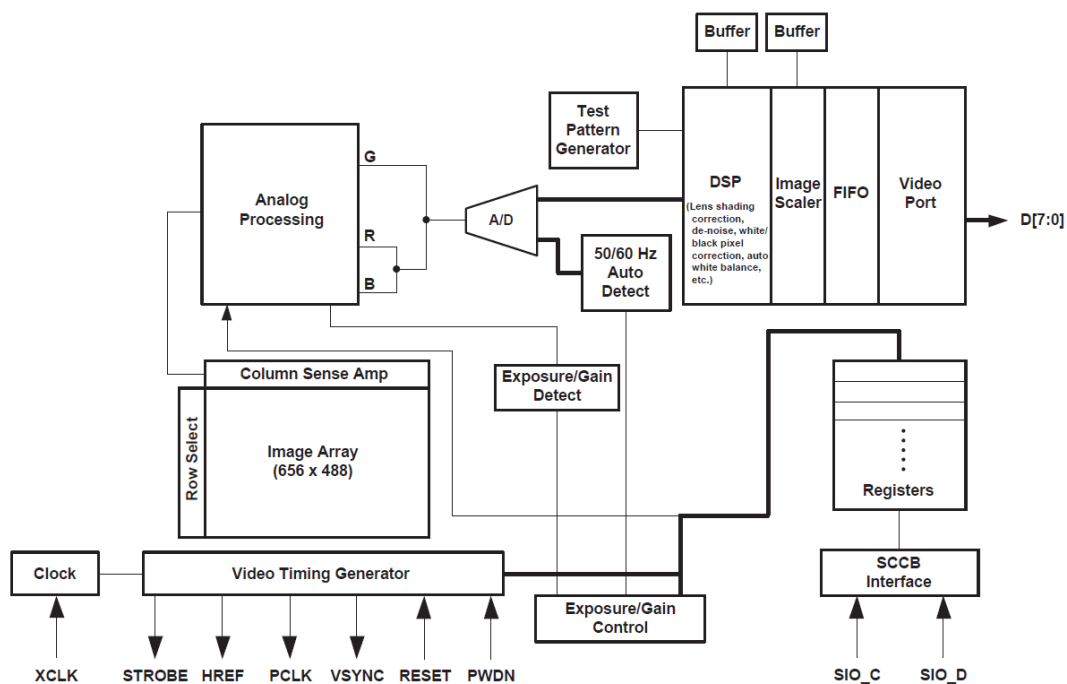
3.Цифрова відеокамера OV7670

Основні параметри

Роздільна здатність	640x480 (VGA)
Формати виводу	YUV/YCbCr 4:2:2 RGD565/555 GRB 4:2:2 Raw RGB Data
Максимальна швидкість передачі	30 fps (VGA)
Об'єктив	1/6"
Кут зору	24 градуси
Живлення	3.3 V



Зовнішній вигляд цифрової відеокамери OV7670

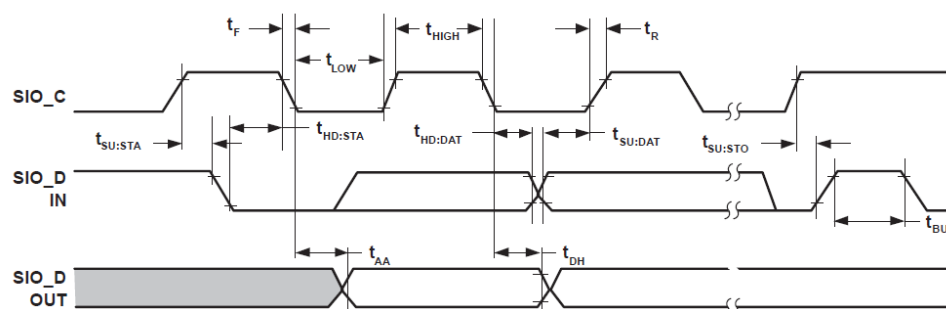


Структурна схема цифрової відеокамери OV7670

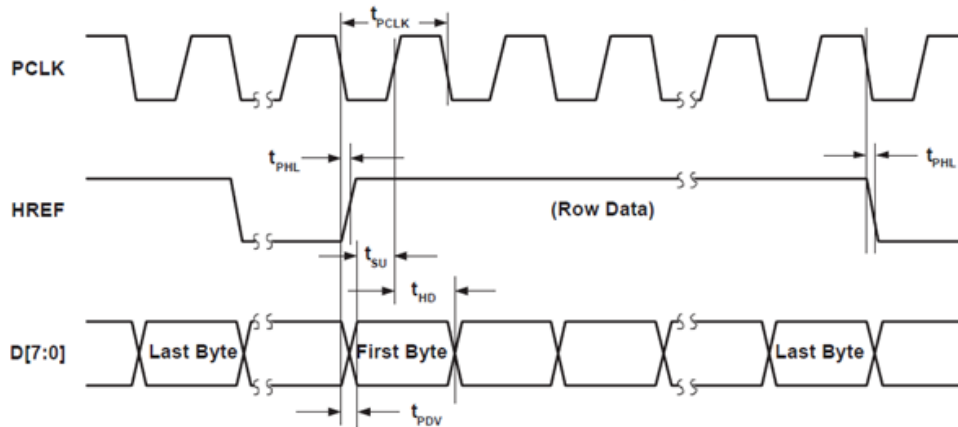
Контакти підключення

1	3.3 V	2	GND	3	SCL	4	SDA
5	VSYNC	6	HREF	7	PCLK	8	XCLK
9	D7	10	D6	11	D5	12	D4
13	D3	14	D2	15	D1	16	D0
17	RESET	18	PWON				

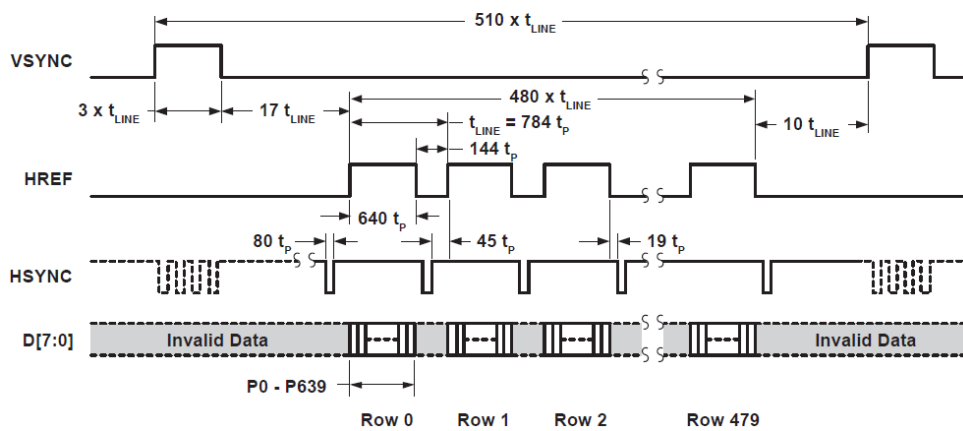
SCCB Timing Diagram



Horizontal Timing



VGA Frame



Діаграми основних режимів

Підключення цифрової відеокамери **OV7670** до контактів **STM32F429DISCOVERY**

<i>PB7</i>	VSYNC	<i>PB6</i>	D5
<i>PA4</i>	HREF	<i>PE4</i>	D4
<i>PA6</i>	PCLK	<i>PC9</i>	D3
<i>PA8</i>	XCLK	<i>PC8</i>	D2
<i>PE6</i>	D7	<i>PC7</i>	D1
<i>PE5</i>	D6	<i>PC6</i>	D0

<i>PB8</i>	SCL (SI0C)
<i>PB9</i>	SDA (SI0D)

ЛІСТИНГИ ПРОГРАМНИХ МОДУЛІВ КЕРУВАННЯ ЦИФРОВОЮ ВІДЕОКАМЕРОЮ OV7670 ТА КОЛЬОРОВИИМ ГРАФІЧНИМ ІНДИКАТОРОМ ІЛІ9341

Програмні модулі для керування цифровою відеокамерою OV7670

```

/*
* CAMERA WIRING:
* 3V3      -      3V      ;      GND      -      GND
* SIOC      -      PB8      ;      SIOD      -      PB9
* VSYNC      -      PB7      ;      HREF      -      PA4
* PCLK      -      PA6      ;      XCLK      -      PA8
* D7      -      PE6      ;      D6      -      PE5
* D5      -      PB6      ;      D4      -      PE4
* D3      -      PC9      ;      D2      -      PC8
* D1      -      PC7      ;      D0      -      PC6
* RESET      -      -----      ;      PWDN      -      -----
*
* =====
*/

#include "OV7670_CONTROL.H"

// IMAGE BUFFER
VOLATILE UINT16_T FRAME_BUFFER[IMG_ROWS*IMG_COLUMNS];

CONST UINT8_T OV7670_REG [OV7670_REG_NUM][2] = {
    {0X12, 0X80},      //RESET REGISTERS

    // IMAGE FORMAT
    {0X12, 0X14},      //QVGA SIZE, RGB MODE
    {0X40, 0XD0},      //RGB565
    {0XB0, 0X84},      //COLOR MODE

    // HARDWARE WINDOW
    {0X11, 0X01},      //PCLK SETTINGS, 15FPS
    {0X32, 0X80},      //HREF
    {0X17, 0X17},      //HSTART
    {0X18, 0X05},      //HSTOP
    {0X03, 0X0A},      //VREF
    {0X19, 0X02},      //VSTART
    {0X1A, 0X7A},      //VSTOP

    // SCALLING NUMBERS
    {0X70, 0X3A},      //X_SCALING
    {0X71, 0X35},      //Y_SCALING
    {0X72, 0X11},      //DCW_SCALING
    {0X73, 0XF0},      //PCLK_DIV_SCALING
    {0XA2, 0X02},      //PCLK_DELAY_SCALING

    // MATRIX COEFFICIENTS
    {0X4F, 0X80},      {0X50, 0X80},
    {0X51, 0X00},      {0X52, 0X22},
    {0X53, 0X5E},      {0X54, 0X80},
    {0X58, 0X9E},

    // GAMMA CURVE VALUES
    {0X7A, 0X20},      {0X7B, 0X10},
    {0X7C, 0X1E},      {0X7D, 0X35},
    {0X7E, 0X5A},      {0X7F, 0X69},
    {0X80, 0X76},      {0X81, 0X80},
    {0X82, 0X88},      {0X83, 0X8F},

```

```

{0X84, 0X96},      {0X85, 0XA3},
{0X86, 0XAF},      {0X87, 0XC4},
{0X88, 0XD7},      {0X89, 0XE8},

```

``` // AGC AND AEC PARAMETERS ```

```

{0XA5, 0X05},      {0XAB, 0X07},
{0X24, 0X95},      {0X25, 0X33},
{0X26, 0XE3},      {0X9F, 0X78},
{0XA0, 0X68},      {0XA1, 0X03},
{0XA6, 0XD8},      {0XA7, 0XD8},
{0XA8, 0XF0},      {0XA9, 0X90},
{0XAA, 0X94},      {0X10, 0X00},

```

``` // AWB PARAMETERS ```

```

{0X43, 0X0A},      {0X44, 0XF0},
{0X45, 0X34},      {0X46, 0X58},
{0X47, 0X28},      {0X48, 0X3A},
{0X59, 0X88},      {0X5A, 0X88},
{0X5B, 0X44},      {0X5C, 0X67},
{0X5D, 0X49},      {0X5E, 0X0E},
{0X6C, 0X0A},      {0X6D, 0X55},
{0X6E, 0X11},      {0X6F, 0X9F},
{0X6A, 0X40},      {0X01, 0X40},
{0X02, 0X60},      {0X13, 0XE7},

```

``` // ADDITIONAL PARAMETERS ```

```

{0X34, 0X11},      {0X3F, 0X00},
{0X75, 0X05},      {0X76, 0XE1},
{0X4C, 0X00},      {0X77, 0X01},
{0XB8, 0X0A},      {0X41, 0X18},
{0X3B, 0X12},      {0XA4, 0X88},
{0X96, 0X00},      {0X97, 0X30},
{0X98, 0X20},      {0X99, 0X30},
{0X9A, 0X84},      {0X9B, 0X29},
{0X9C, 0X03},      {0X9D, 0X4C},
{0X9E, 0X3F},      {0X78, 0X04},
{0X0E, 0X61},      {0X0F, 0X4B},
{0X16, 0X02},      {0X1E, 0X00},
{0X21, 0X02},      {0X22, 0X91},
{0X29, 0X07},      {0X33, 0X0B},
{0X35, 0X0B},      {0X37, 0X1D},
{0X38, 0X71},      {0X39, 0X2A},
{0X3C, 0X78},      {0X4D, 0X40},
{0X4E, 0X20},      {0X69, 0X00},
{0X6B, 0X3A},      {0X74, 0X10},
{0X8D, 0X4F},      {0X8E, 0X00},
{0X8F, 0X00},      {0X90, 0X00},
{0X91, 0X00},      {0X96, 0X00},
{0X9A, 0X00},      {0XB1, 0X0C},
{0XB2, 0X0E},      {0XB3, 0X82},
{0X4B, 0X01},

```

```

};
VOID DELAY(VOLATILE UINT16_T NCOUNT);
VOID DELAY(VOLATILE UINT16_T NCOUNT)
{
    WHILE(NCOUNT--){
    }
}

VOID MCO1_INIT(VOID){
    GPIO_INITTYPEDEF GPIO_INITSTRUCTURE;

    RCC_CLOCKSECURITYSYSTEMCMD(ENABLE);

```

```

RCC_AHB1PERIPHCLKCMD(RCC_AHB1PERIPH_GPIOA, ENABLE);

// GPIO CONFIG
GPIO_INITSTRUCTURE.GPIO_PIN = GPIO_PIN_8;           //PA8 - XCLK
GPIO_INITSTRUCTURE.GPIO_SPEED = GPIO_SPEED_100MHZ;
GPIO_INITSTRUCTURE.GPIO_MODE = GPIO_MODE_AF;
GPIO_INITSTRUCTURE.GPIO_OTYPE = GPIO_OTYPE_PP;
GPIO_INITSTRUCTURE.GPIO_PUPD = GPIO_PUPD_UP;
GPIO_INIT(GPIOA, &GPIO_INITSTRUCTURE);

// GPIO AF CONFIG
GPIO_PINAFCONFIG(GPIOA, GPIO_PINSOURCE8, GPIO_AF_MCO);

// MCO CLOCK SOURCE
RCC_MCO1CONFIG(RCC_MCO1SOURCE_PLLCLK, RCC_MCO1DIV_4);
}

VOID SCCB_INIT(VOID){
    GPIO_INITTYPEDEF GPIO_INITSTRUCTURE;
    I2C_INITTYPEDEF I2C_INITSTRUCTURE;

    RCC_APB1PERIPHCLKCMD(RCC_APB1PERIPH_I2C1, ENABLE);
    RCC_AHB1PERIPHCLKCMD(RCC_AHB1PERIPH_GPIOB, ENABLE);

    // GPIO CONFIG
    GPIO_INITSTRUCTURE.GPIO_PIN = GPIO_PIN_8 | GPIO_PIN_9;           //PB8 - SIOC

    //PB9 - SIOD
    GPIO_INITSTRUCTURE.GPIO_MODE = GPIO_MODE_AF;
    GPIO_INITSTRUCTURE.GPIO_SPEED = GPIO_SPEED_2MHZ;
    GPIO_INITSTRUCTURE.GPIO_OTYPE = GPIO_OTYPE_OD;
    GPIO_INITSTRUCTURE.GPIO_PUPD = GPIO_PUPD_UP;
    GPIO_INIT(GPIOB, &GPIO_INITSTRUCTURE);

    // GPIO AF CONFIG
    GPIO_PINAFCONFIG(GPIOB, GPIO_PINSOURCE8, GPIO_AF_I2C1);
    GPIO_PINAFCONFIG(GPIOB, GPIO_PINSOURCE9, GPIO_AF_I2C1);

    // I2C CONFIG
    I2C_DEINIT(I2C1);
    I2C_INITSTRUCTURE.I2C_MODE = I2C_MODE_I2C;
    I2C_INITSTRUCTURE.I2C_DUTYCYCLE = I2C_DUTYCYCLE_2;
    I2C_INITSTRUCTURE.I2C_OWNADDRESS1 = 0X00;
    I2C_INITSTRUCTURE.I2C_ACK = I2C_ACK_ENABLE;
    I2C_INITSTRUCTURE.I2C_ACKNOWLEDGEDADDRESS = I2C_ACKNOWLEDGEDADDRESS_7BIT;
    I2C_INITSTRUCTURE.I2C_CLOCKSPEED = 100000;
    I2C_ITCONFIG(I2C1, I2C_IT_ERR, ENABLE);
    I2C_INIT(I2C1, &I2C_INITSTRUCTURE);
    I2C_CMD(I2C1, ENABLE);
}

BOOL SCCB_WRITE_REG(UINT8_T REG_ADDR, UINT8_T* DATA);
BOOL SCCB_WRITE_REG(UINT8_T REG_ADDR, UINT8_T* DATA)
{
    UINT32_T TIMEOUT = 0X7FFFFFFF;

    WHILE(I2C_GETFLAGSTATUS(I2C1, I2C_FLAG_BUSY)){
        IF ((TIMEOUT--) == 0){
            RETURN TRUE;
        }
    }
}

```

```

// SEND START BIT
I2C_GENERATESTART(I2C1, ENABLE);

WHILE( !I2C_CHECKEVENT(I2C1,I2C_EVENT_MASTER_MODE_SELECT)){
    IF ((TIMEOUT--) == 0){
        RETURN TRUE;
    }
}

// SEND SLAVE ADDRESS (CAMERA WRITE ADDRESS)
I2C_SEND7BITADDRESS(I2C1, OV7670_WRITE_ADDR, I2C_DIRECTION_TRANSMITTER);

WHILE(!I2C_CHECKEVENT(I2C1, I2C_EVENT_MASTER_TRANSMITTER_MODE_SELECTED)){
    IF ((TIMEOUT--) == 0){
        RETURN TRUE;
    }
}

// SEND REGISTER ADDRESS
I2C_SENDDATA(I2C1, REG_ADDR);

WHILE(!I2C_CHECKEVENT(I2C1,I2C_EVENT_MASTER_BYTE_TRANSMITTED)){
    IF ((TIMEOUT--) == 0){
        RETURN TRUE;
    }
}

// SEND NEW REGISTER VALUE
I2C_SENDDATA(I2C1, *DATA);

WHILE(!I2C_CHECKEVENT(I2C1,I2C_EVENT_MASTER_BYTE_TRANSMITTED)){
    IF ((TIMEOUT--) == 0){
        RETURN TRUE;
    }
}

// SEND STOP BIT
I2C_GENERATESTOP(I2C1, ENABLE);
RETURN FALSE;
}

BOOL OV7670_INIT(VOID){
    UINT8_T DATA, I = 0;
    BOOL ERR;

    // CONFIGURE CAMERA REGISTERS
    FOR(I=0; I<OV7670_REG_NUM ;I++){
        DATA = OV7670_REG[I][1];
        ERR = SCCB_WRITE_REG(OV7670_REG[I][0], &DATA);

        IF (ERR == TRUE)
            BREAK;

        LCD_ILI9341_DRAWLINE(99+I, 110, 99+I, 130, ILI9341_COLOR_WHITE);
        DELAY(0XFFFF);
    }

    RETURN ERR;
}

VOID DCMI_DMA_INIT(VOID){
    GPIO_INITTYPEDEF GPIO_INITSTRUCTURE;

```



```

DCMI_INITTYPEDEF DCMI_INITSTRUCTURE;
DMA_INITTYPEDEF DMA_INITSTRUCTURE;
NVIC_INITTYPEDEF NVIC_INITSTRUCTURE;

RCC_AHB2PERIPHCLKCMD(RCC_AHB2PERIPH_DCMI, ENABLE);
RCC_AHB1PERIPHCLKCMD(RCC_AHB1PERIPH_GPIOA, ENABLE);
RCC_AHB1PERIPHCLKCMD(RCC_AHB1PERIPH_GPIOB, ENABLE);
RCC_AHB1PERIPHCLKCMD(RCC_AHB1PERIPH_GPIOC, ENABLE);
RCC_AHB1PERIPHCLKCMD(RCC_AHB1PERIPH_GPIOE, ENABLE);
RCC_AHB1PERIPHCLKCMD(RCC_AHB1PERIPH_DMA2, ENABLE);

// GPIO CONFIG
GPIO_INITSTRUCTURE.GPIO_PIN = GPIO_PIN_4 | GPIO_PIN_6;           //PA4 - HREF

//PA6 - PCLK
GPIO_INITSTRUCTURE.GPIO_MODE = GPIO_MODE_AF;
GPIO_INITSTRUCTURE.GPIO_SPEED = GPIO_SPEED_100MHZ;
GPIO_INITSTRUCTURE.GPIO_OTYPE = GPIO_OTYPE_PP;
GPIO_INITSTRUCTURE.GPIO_PUPD = GPIO_PUPD_UP ;
GPIO_INIT(GPIOA, &GPIO_INITSTRUCTURE);

GPIO_INITSTRUCTURE.GPIO_PIN = GPIO_PIN_6 | GPIO_PIN_7;           //PB6 - D5

//PB7 - VSYNC
GPIO_INITSTRUCTURE.GPIO_MODE = GPIO_MODE_AF;
GPIO_INITSTRUCTURE.GPIO_SPEED = GPIO_SPEED_100MHZ;
GPIO_INITSTRUCTURE.GPIO_OTYPE = GPIO_OTYPE_PP;
GPIO_INITSTRUCTURE.GPIO_PUPD = GPIO_PUPD_UP ;
GPIO_INIT(GPIOB, &GPIO_INITSTRUCTURE);

GPIO_INITSTRUCTURE.GPIO_PIN = GPIO_PIN_6 | GPIO_PIN_7 | GPIO_PIN_8 | GPIO_PIN_9;           //PC6 - D0

//PC7 - D1

//PC8 - D2

//PC9 - D3
GPIO_INITSTRUCTURE.GPIO_MODE = GPIO_MODE_AF;
GPIO_INITSTRUCTURE.GPIO_SPEED = GPIO_SPEED_100MHZ;
GPIO_INITSTRUCTURE.GPIO_OTYPE = GPIO_OTYPE_PP;
GPIO_INITSTRUCTURE.GPIO_PUPD = GPIO_PUPD_UP ;
GPIO_INIT(GPIOC, &GPIO_INITSTRUCTURE);

GPIO_INITSTRUCTURE.GPIO_PIN = GPIO_PIN_4 | GPIO_PIN_5 | GPIO_PIN_6;           //PE4 - D4

//PE5 - D6

//PE6 - D7
GPIO_INITSTRUCTURE.GPIO_MODE = GPIO_MODE_AF;
GPIO_INITSTRUCTURE.GPIO_SPEED = GPIO_SPEED_100MHZ;
GPIO_INITSTRUCTURE.GPIO_OTYPE = GPIO_OTYPE_PP;
GPIO_INITSTRUCTURE.GPIO_PUPD = GPIO_PUPD_UP ;
GPIO_INIT(GPIOE, &GPIO_INITSTRUCTURE);

```

```

// GPIO AF CONFIG
GPIO_PINAFCONFIG(GPIOB, GPIO_PINSOURCE7, GPIO_AF_DCM1);
GPIO_PINAFCONFIG(GPIOA, GPIO_PINSOURCE4, GPIO_AF_DCM1);
GPIO_PINAFCONFIG(GPIOA, GPIO_PINSOURCE6, GPIO_AF_DCM1);
GPIO_PINAFCONFIG(GPIOC, GPIO_PINSOURCE6, GPIO_AF_DCM1);
GPIO_PINAFCONFIG(GPIOC, GPIO_PINSOURCE7, GPIO_AF_DCM1);
GPIO_PINAFCONFIG(GPIOC, GPIO_PINSOURCE8, GPIO_AF_DCM1);
GPIO_PINAFCONFIG(GPIOC, GPIO_PINSOURCE9, GPIO_AF_DCM1);
GPIO_PINAFCONFIG(GPIOE, GPIO_PINSOURCE4, GPIO_AF_DCM1);
GPIO_PINAFCONFIG(GPIOB, GPIO_PINSOURCE6, GPIO_AF_DCM1);
GPIO_PINAFCONFIG(GPIOE, GPIO_PINSOURCE5, GPIO_AF_DCM1);
GPIO_PINAFCONFIG(GPIOE, GPIO_PINSOURCE6, GPIO_AF_DCM1);

// DCM1 CONFIG
DCM1_DEINIT();
DCM1_INITSTRUCTURE.DCM1_CAPTUREMODE = DCM1_CAPTUREMODE_CONTINUOUS;
DCM1_INITSTRUCTURE.DCM1_EXTENDEDATAMODE = DCM1_EXTENDEDATAMODE_8B;
DCM1_INITSTRUCTURE.DCM1_CAPTURERATE = DCM1_CAPTURERATE_ALL_FRAME;
DCM1_INITSTRUCTURE.DCM1_PCKPOLARITY = DCM1_PCKPOLARITY_RISING;
DCM1_INITSTRUCTURE.DCM1_HSPOLARITY = DCM1_HSPOLARITY_LOW;
DCM1_INITSTRUCTURE.DCM1_VSPOLARITY = DCM1_VSPOLARITY_HIGH;
DCM1_INITSTRUCTURE.DCM1_SYNCHROMODE = DCM1_SYNCHROMODE_HARDWARE;
DCM1_INIT(&DCM1_INITSTRUCTURE);
DCM1_CMD(ENABLE);

// DMA CONFIG
DMA_DEINIT(DMA2_STREAM1);
DMA_INITSTRUCTURE.DMA_CHANNEL = DMA_CHANNEL_1;
DMA_INITSTRUCTURE.DMA_PERIPHERALBASEADDR = (UINT32_T)&DCM1->DR;
DMA_INITSTRUCTURE.DMA_MEMORY0BASEADDR = (UINT32_T)FRAME_BUFFER;
DMA_INITSTRUCTURE.DMA_DIR = DMA_DIR_PERIPHERALTOMEMORY;
DMA_INITSTRUCTURE.DMA_BUFFERSIZE = IMG_ROWS*IMG_COLUMNS*2/4;
DMA_INITSTRUCTURE.DMA_PERIPHERALINC = DMA_PERIPHERALINC_DISABLE;
DMA_INITSTRUCTURE.DMA_MEMORYINC = DMA_MEMORYINC_ENABLE;
DMA_INITSTRUCTURE.DMA_PERIPHERALDATASIZE = DMA_PERIPHERALDATASIZE_WORD;
DMA_INITSTRUCTURE.DMA_MEMORYDATASIZE = DMA_MEMORYDATASIZE_HALFWORD;
DMA_INITSTRUCTURE.DMA_MODE = DMA_MODE_NORMAL;
DMA_INITSTRUCTURE.DMA_PRIORITY = DMA_PRIORITY_HIGH;
DMA_INITSTRUCTURE.DMA_FIFOMODE = DMA_FIFOMODE_ENABLE;
DMA_INITSTRUCTURE.DMA_FIFOTHRESHOLD = DMA_FIFOTHRESHOLD_FULL;
DMA_INITSTRUCTURE.DMA_MEMORYBURST = DMA_MEMORYBURST_SINGLE;
DMA_INITSTRUCTURE.DMA_PERIPHERALBURST = DMA_PERIPHERALBURST_SINGLE;
DMA_INIT(DMA2_STREAM1, &DMA_INITSTRUCTURE);
DMA_CMD(DMA2_STREAM1, ENABLE);

// DMA INTERRUPT
DMA_ITCONFIG(DMA2_STREAM1, DMA_IT_TC, ENABLE);

NVIC_INITSTRUCTURE.NVIC_IRQCHANNEL = DMA2_STREAM1_IRQN;
NVIC_INITSTRUCTURE.NVIC_IRQCHANNELPREEMPTIONPRIORITY = 0;
NVIC_INITSTRUCTURE.NVIC_IRQCHANNELSUBPRIORITY = 0;
NVIC_INITSTRUCTURE.NVIC_IRQCHANNELCMD = ENABLE;
NVIC_INIT(&NVIC_INITSTRUCTURE);
}

```

Програмні модулі для керування кольоровим графічним індикатором ILI9341

```
#include "lcd_ili9341.h"

uint16_t ILI9341_x;
uint16_t ILI9341_y;
LCD_ILI9341_Options_t ILI9341_Opts;
uint8_t ILI9341_INT_CalledFromPuts = 0;

void LCD_ILI9341_Init() {
    GPIO_InitTypeDef GPIO_InitDef;

    RCC_AHB1PeriphClockCmd(ILI9341_WRX_CLK, ENABLE);
    GPIO_InitDef.GPIO_Pin = ILI9341_WRX_PIN;
    GPIO_InitDef.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_InitDef.GPIO_Mode = GPIO_Mode_OUT;
    GPIO_InitDef.GPIO_OType = GPIO_OType_PP;
    GPIO_InitDef.GPIO_PuPd = GPIO_PuPd_NOPULL;
    GPIO_Init(ILI9341_WRX_PORT, &GPIO_InitDef);

    RCC_AHB1PeriphClockCmd(ILI9341_CS_CLK, ENABLE);
    GPIO_InitDef.GPIO_Pin = ILI9341_CS_PIN;
    GPIO_Init(ILI9341_CS_PORT, &GPIO_InitDef);

    RCC_AHB1PeriphClockCmd(ILI9341_RST_CLK, ENABLE);
    GPIO_InitDef.GPIO_PuPd = GPIO_PuPd_UP;
    GPIO_InitDef.GPIO_Pin = ILI9341_RST_PIN;
    GPIO_Init(ILI9341_RST_PORT, &GPIO_InitDef);

    ILI9341_CS_SET;

    LCD_SPI_Init();

    LCD_ILI9341_InitLCD();

    ILI9341_x = ILI9341_y = 0;

    ILI9341_Opts.width = ILI9341_WIDTH;
    ILI9341_Opts.height = ILI9341_HEIGHT;
    ILI9341_Opts.orientation = LCD_ILI9341_Portrait;
}

void LCD_ILI9341_InitLCD(void) {
    // ILI9341_RST_RESET;
    ILI9341_RST_SET;

    LCD_ILI9341_SendCommand(ILI9341_RESET);
    LCD_ILI9341_Delay(2000000);

    LCD_ILI9341_SendCommand(ILI9341_POWERA);
    LCD_ILI9341_SendData(0x39);
    LCD_ILI9341_SendData(0x2C);
    LCD_ILI9341_SendData(0x00);
    LCD_ILI9341_SendData(0x34);
    LCD_ILI9341_SendData(0x02);
    LCD_ILI9341_SendCommand(ILI9341_POWERB);
    LCD_ILI9341_SendData(0x00);
    LCD_ILI9341_SendData(0xC1);
    LCD_ILI9341_SendData(0x30);
    LCD_ILI9341_SendCommand(ILI9341_DTCA);
    LCD_ILI9341_SendData(0x85);
    LCD_ILI9341_SendData(0x00);
```

```

LCD_ILI9341_SendData(0x78);
LCD_ILI9341_SendCommand(ILI9341_DTCB);
LCD_ILI9341_SendData(0x00);
LCD_ILI9341_SendData(0x00);
LCD_ILI9341_SendCommand(ILI9341_POWER_SEQ);
LCD_ILI9341_SendData(0x64);
LCD_ILI9341_SendData(0x03);
LCD_ILI9341_SendData(0x12);
LCD_ILI9341_SendData(0x81);
LCD_ILI9341_SendCommand(ILI9341_PRC);
LCD_ILI9341_SendData(0x20);
LCD_ILI9341_SendCommand(ILI9341_POWER1);
LCD_ILI9341_SendData(0x23);
LCD_ILI9341_SendCommand(ILI9341_POWER2);
LCD_ILI9341_SendData(0x10);
LCD_ILI9341_SendCommand(ILI9341_VCOM1);
LCD_ILI9341_SendData(0x3E);
LCD_ILI9341_SendData(0x28);
LCD_ILI9341_SendCommand(ILI9341_VCOM2);
LCD_ILI9341_SendData(0x86);
LCD_ILI9341_SendCommand(ILI9341_MAC);
LCD_ILI9341_SendData(0x48);
LCD_ILI9341_SendCommand(ILI9341_PIXEL_FORMAT);
LCD_ILI9341_SendData(0x55);
LCD_ILI9341_SendCommand(ILI9341_FRC);
LCD_ILI9341_SendData(0x00);
LCD_ILI9341_SendData(0x18);
LCD_ILI9341_SendCommand(ILI9341_DFC);
LCD_ILI9341_SendData(0x08);
LCD_ILI9341_SendData(0x82);
LCD_ILI9341_SendData(0x27);
LCD_ILI9341_SendCommand(ILI9341_3GAMMA_EN);
LCD_ILI9341_SendData(0x00);
LCD_ILI9341_SendCommand(ILI9341_COLUMN_ADDR);
LCD_ILI9341_SendData(0x00);
LCD_ILI9341_SendData(0x00);
LCD_ILI9341_SendData(0x00);
LCD_ILI9341_SendData(0xEF);
LCD_ILI9341_SendCommand(ILI9341_PAGE_ADDR);
LCD_ILI9341_SendData(0x00);
LCD_ILI9341_SendData(0x00);
LCD_ILI9341_SendData(0x01);
LCD_ILI9341_SendData(0x3F);
LCD_ILI9341_SendCommand(ILI9341_GAMMA);
LCD_ILI9341_SendData(0x01);
LCD_ILI9341_SendCommand(ILI9341_PGAMMA);
LCD_ILI9341_SendData(0x0F);
LCD_ILI9341_SendData(0x31);
LCD_ILI9341_SendData(0x2B);
LCD_ILI9341_SendData(0x0C);
LCD_ILI9341_SendData(0x0E);
LCD_ILI9341_SendData(0x08);
LCD_ILI9341_SendData(0x4E);
LCD_ILI9341_SendData(0xF1);
LCD_ILI9341_SendData(0x37);
LCD_ILI9341_SendData(0x07);
LCD_ILI9341_SendData(0x10);
LCD_ILI9341_SendData(0x03);
LCD_ILI9341_SendData(0x0E);
LCD_ILI9341_SendData(0x09);
LCD_ILI9341_SendData(0x00);
LCD_ILI9341_SendCommand(ILI9341_NGAMMA);
LCD_ILI9341_SendData(0x00);

```

```

LCD_ILI9341_SendData(0x0E);
LCD_ILI9341_SendData(0x14);
LCD_ILI9341_SendData(0x03);
LCD_ILI9341_SendData(0x11);
LCD_ILI9341_SendData(0x07);
LCD_ILI9341_SendData(0x31);
LCD_ILI9341_SendData(0xC1);
LCD_ILI9341_SendData(0x48);
LCD_ILI9341_SendData(0x08);
LCD_ILI9341_SendData(0x0F);
LCD_ILI9341_SendData(0x0C);
LCD_ILI9341_SendData(0x31);
LCD_ILI9341_SendData(0x36);
LCD_ILI9341_SendData(0x0F);
LCD_ILI9341_SendCommand(ILI9341_SLEEP_OUT);

LCD_ILI9341_Delay(1000000);

LCD_ILI9341_SendCommand(ILI9341_DISPLAY_ON);
LCD_ILI9341_SendCommand(ILI9341_GRAM);
}

void LCD_ILI9341_SendCommand(uint8_t data) {
    ILI9341_WRX_RESET;
    ILI9341_CS_RESET;
    LCD_SPI_Send(ILI9341_SPI, data);
    ILI9341_CS_SET;
}

void LCD_ILI9341_SendData(uint8_t data) {
    ILI9341_WRX_SET;
    ILI9341_CS_RESET;
    LCD_SPI_Send(ILI9341_SPI, data);
    ILI9341_CS_SET;
}

void LCD_ILI9341_DrawPixel(uint16_t x, uint16_t y, uint16_t color) {
    LCD_ILI9341_SetCursorPosition(x, y, x, y);

    LCD_ILI9341_SendCommand(ILI9341_GRAM);
    LCD_ILI9341_SendData(color >> 8);
    LCD_ILI9341_SendData(color & 0xFF);
}

void LCD_ILI9341_SetCursorPosition(uint16_t x1, uint16_t y1, uint16_t x2, uint16_t y2) {
    LCD_ILI9341_SendCommand(ILI9341_COLUMN_ADDR);
    LCD_ILI9341_SendData(x1 >> 8);
    LCD_ILI9341_SendData(x1 & 0xFF);
    LCD_ILI9341_SendData(x2 >> 8);
    LCD_ILI9341_SendData(x2 & 0xFF);

    LCD_ILI9341_SendCommand(ILI9341_PAGE_ADDR);
    LCD_ILI9341_SendData(y1 >> 8);
    LCD_ILI9341_SendData(y1 & 0xFF);
    LCD_ILI9341_SendData(y2 >> 8);
    LCD_ILI9341_SendData(y2 & 0xFF);
}

void LCD_ILI9341_Fill(uint16_t color) {
    unsigned int n, i, j;
    i = color >> 8;

```

```

        j = color & 0xFF;
        LCD_ILI9341_SetCursorPosition(0, 0, ILI9341_Opts.width - 1, ILI9341_Opts.height - 1);

        LCD_ILI9341_SendCommand(ILI9341_GRAM);

        for (n = 0; n < ILI9341_PIXEL; n++) {
            LCD_ILI9341_SendData(i);
            LCD_ILI9341_SendData(j);
        }
    }

void LCD_ILI9341_DisplayImage(uint16_t image[ILI9341_PIXEL]) {
    uint32_t n, i, j;

    LCD_ILI9341_SetCursorPosition(0, 0, ILI9341_Opts.width - 1, ILI9341_Opts.height - 1);

    LCD_ILI9341_SendCommand(ILI9341_GRAM);

    ILI9341_WRX_SET;
    ILI9341_CS_RESET;

    for (n = 0; n < ILI9341_PIXEL; n++) {
        i = image[n] >> 8;
        j = image[n] & 0xFF;

        LCD_SPI_Send(ILI9341_SPI, i);
        LCD_SPI_Send(ILI9341_SPI, j);
    }

    ILI9341_CS_SET;
}

void LCD_ILI9341_Delay(volatile unsigned int delay) {
    for (; delay != 0; delay--);
}

void LCD_ILI9341_Rotate(LCD_ILI9341_Orientation_t orientation) {
    LCD_ILI9341_SendCommand(ILI9341_MAC);
    if (orientation == LCD_ILI9341_Orientation_Portrait_1) {
        LCD_ILI9341_SendData(0x58);
    } else if (orientation == LCD_ILI9341_Orientation_Portrait_2) {
        LCD_ILI9341_SendData(0x88);
    } else if (orientation == LCD_ILI9341_Orientation_Landscape_1) {
        LCD_ILI9341_SendData(0x28);
    } else if (orientation == LCD_ILI9341_Orientation_Landscape_2) {
        LCD_ILI9341_SendData(0xE8);
    }

    if (orientation == LCD_ILI9341_Orientation_Portrait_1 || orientation == LCD_ILI9341_Orientation_Portrait_2) {
        ILI9341_Opts.width = ILI9341_WIDTH;
        ILI9341_Opts.height = ILI9341_HEIGHT;
        ILI9341_Opts.orientation = LCD_ILI9341_Portrait;
    } else {
        ILI9341_Opts.width = ILI9341_HEIGHT;
        ILI9341_Opts.height = ILI9341_WIDTH;
        ILI9341_Opts.orientation = LCD_ILI9341_Landscape;
    }
}

void LCD_ILI9341_Puts(uint16_t x, uint16_t y, char *str, LCD_FontDef_t *font, uint16_t foreground, uint16_t background) {
    uint16_t startX = x;

    /* Set X and Y coordinates */

```

```

ILI9341_x = x;
ILI9341_y = y;

while (*str) {
    //New line
    if (*str == '\n') {
        ILI9341_y += font->FontHeight + 1;
        //if after \n is also \r, than go to the left of the screen
        if (*(str + 1) == '\r') {
            ILI9341_x = 0;
            str++;
        } else {
            ILI9341_x = startX;
        }
        str++;
        continue;
    } else if (*str == '\r') {
        str++;
        continue;
    }

    LCD_ILI9341_Putc(ILI9341_x, ILI9341_y, *str++, font, foreground, background);
}

}

void LCD_ILI9341_GetStringSize(char *str, LCD_FontDef_t *font, uint16_t *width, uint16_t *height) {
    uint16_t w = 0;
    *height = font->FontHeight;
    while (*str++) {
        w += font->FontWidth;
    }
    *width = w;
}

void LCD_ILI9341_Putc(uint16_t x, uint16_t y, char c, LCD_FontDef_t *font, uint16_t foreground, uint16_t background) {
    uint32_t i, b, j;
    /* Set coordinates */
    ILI9341_x = x;
    ILI9341_y = y;
    if ((ILI9341_x + font->FontWidth) > ILI9341_Opts.width) {
        //If at the end of a line of display, go to new line and set x to 0 position
        ILI9341_y += font->FontHeight;
        ILI9341_x = 0;
    }
    for (i = 0; i < font->FontHeight; i++) {
        b = font->data[(c - 32) * font->FontHeight + i];
        for (j = 0; j < font->FontWidth; j++) {
            if ((b << j) & 0x8000) {
                LCD_ILI9341_DrawPixel(ILI9341_x + j, (ILI9341_y + i), foreground);
            } else {
                LCD_ILI9341_DrawPixel(ILI9341_x + j, (ILI9341_y + i), background);
            }
        }
    }
    ILI9341_x += font->FontWidth;
}

void LCD_ILI9341_DrawLine(uint16_t x0, uint16_t y0, uint16_t x1, uint16_t y1, uint16_t color) {
    /* Code by dewoller: https://github.com/dewoller */

    int16_t dx, dy, sx, sy, err, e2;

```



```

/* Check for overflow */
if (x0 >= ILI9341_Opts.width) {
    x0 = ILI9341_Opts.width - 1;
}
if (x1 >= ILI9341_Opts.width) {
    x1 = ILI9341_Opts.width - 1;
}
if (y0 >= ILI9341_Opts.height) {
    y0 = ILI9341_Opts.height - 1;
}
if (y1 >= ILI9341_Opts.height) {
    y1 = ILI9341_Opts.height - 1;
}

dx = (x0 < x1) ? (x1 - x0) : (x0 - x1);
dy = (y0 < y1) ? (y1 - y0) : (y0 - y1);
sx = (x0 < x1) ? 1 : -1;
sy = (y0 < y1) ? 1 : -1;
err = ((dx > dy) ? dx : -dy) / 2;

while (1) {
    LCD_ILI9341_DrawPixel(x0, y0, color);
    if (x0 == x1 && y0 == y1) {
        break;
    }
    e2 = err;
    if (e2 > -dx) {
        err -= dy;
        x0 += sx;
    }
    if (e2 < dy) {
        err += dx;
        y0 += sy;
    }
}
}

void LCD_ILI9341_DrawRectangle(uint16_t x0, uint16_t y0, uint16_t x1, uint16_t y1, uint16_t color) {
    LCD_ILI9341_DrawLine(x0, y0, x1, y0, color); //Top
    LCD_ILI9341_DrawLine(x0, y0, x0, y1, color); //Left
    LCD_ILI9341_DrawLine(x1, y0, x1, y1, color); //Right
    LCD_ILI9341_DrawLine(x0, y1, x1, y1, color); //Bottom
}

void LCD_ILI9341_DrawFilledRectangle(uint16_t x0, uint16_t y0, uint16_t x1, uint16_t y1, uint16_t color) {
    for (; y0 < y1; y0++) {
        LCD_ILI9341_DrawLine(x0, y0, x1, y0, color);
    }
}

void LCD_ILI9341_DrawCircle(int16_t x0, int16_t y0, int16_t r, uint16_t color) {
    int16_t f = 1 - r;
    int16_t ddF_x = 1;
    int16_t ddF_y = -2 * r;
    int16_t x = 0;
    int16_t y = r;

    LCD_ILI9341_DrawPixel(x0, y0 + r, color);
    LCD_ILI9341_DrawPixel(x0, y0 - r, color);
    LCD_ILI9341_DrawPixel(x0 + r, y0, color);
    LCD_ILI9341_DrawPixel(x0 - r, y0, color);

    while (x < y) {

```

```

    if (f >= 0) {
        y--;
        ddF_y += 2;
        f += ddF_y;
    }
    x++;
    ddF_x += 2;
    f += ddF_x;

    LCD_ILI9341_DrawPixel(x0 + x, y0 + y, color);
    LCD_ILI9341_DrawPixel(x0 - x, y0 + y, color);
    LCD_ILI9341_DrawPixel(x0 + x, y0 - y, color);
    LCD_ILI9341_DrawPixel(x0 - x, y0 - y, color);

    LCD_ILI9341_DrawPixel(x0 + y, y0 + x, color);
    LCD_ILI9341_DrawPixel(x0 - y, y0 + x, color);
    LCD_ILI9341_DrawPixel(x0 + y, y0 - x, color);
    LCD_ILI9341_DrawPixel(x0 - y, y0 - x, color);
}
}

void LCD_ILI9341_DrawFilledCircle(int16_t x0, int16_t y0, int16_t r, uint16_t color) {
    int16_t f = 1 - r;
    int16_t ddF_x = 1;
    int16_t ddF_y = -2 * r;
    int16_t x = 0;
    int16_t y = r;

    LCD_ILI9341_DrawPixel(x0, y0 + r, color);
    LCD_ILI9341_DrawPixel(x0, y0 - r, color);
    LCD_ILI9341_DrawPixel(x0 + r, y0, color);
    LCD_ILI9341_DrawPixel(x0 - r, y0, color);
    LCD_ILI9341_DrawLine(x0 - r, y0, x0 + r, y0, color);

    while (x < y) {
        if (f >= 0) {
            y--;
            ddF_y += 2;
            f += ddF_y;
        }
        x++;
        ddF_x += 2;
        f += ddF_x;

        LCD_ILI9341_DrawLine(x0 - x, y0 + y, x0 + x, y0 + y, color);
        LCD_ILI9341_DrawLine(x0 + x, y0 - y, x0 - x, y0 - y, color);

        LCD_ILI9341_DrawLine(x0 + y, y0 + x, x0 - y, y0 + x, color);
        LCD_ILI9341_DrawLine(x0 + y, y0 - x, x0 - y, y0 - x, color);
    }
}

```

```

#include "lcd_ili9341.h"

```

```

#ifndef LCD_ILI9341_H
#define LCD_ILI9341_H

```

```

// INCLUDES
#include "STM32F4XX.H"

```

```

#include "STM32F4XX_RCC.H"
#include "STM32F4XX_GPIO.H"
#include "LCD_SPI.H"
#include "LCD_FONTS.H"

// SPI SET
#define ILI9341_SPI SPI5

// CS PIN
#define ILI9341_CS_CLK RCC_AHB1PERIPH_GPIOC
#define ILI9341_CS_PORT GPIOC
#define ILI9341_CS_PIN GPIO_PIN_2

// WRX PIN
#define ILI9341_WRX_CLK RCC_AHB1PERIPH_GPIOD
#define ILI9341_WRX_PORT GPIOD
#define ILI9341_WRX_PIN GPIO_PIN_13

// RESET PIN
#define ILI9341_RST_CLK RCC_AHB1PERIPH_GPIOD
#define ILI9341_RST_PORT GPIOD
#define ILI9341_RST_PIN GPIO_PIN_12

#define ILI9341_RST_SET GPIO_SETBITS(ILI9341_RST_PORT, ILI9341_RST_PIN)
#define ILI9341_RST_RESET GPIO_RESETBITS(ILI9341_RST_PORT, ILI9341_RST_PIN)
#define ILI9341_CS_SET GPIO_SETBITS(ILI9341_CS_PORT, ILI9341_CS_PIN)
#define ILI9341_CS_RESET GPIO_RESETBITS(ILI9341_CS_PORT, ILI9341_CS_PIN)
#define ILI9341_WRX_SET GPIO_SETBITS(ILI9341_WRX_PORT, ILI9341_WRX_PIN)
#define ILI9341_WRX_RESET GPIO_RESETBITS(ILI9341_WRX_PORT, ILI9341_WRX_PIN)

// LCD SETTINGS
#define ILI9341_WIDTH 240
#define ILI9341_HEIGHT 320
#define ILI9341_PIXEL 76800

// COLORS
#define ILI9341_COLOR_WHITE 0xFFFF
#define ILI9341_COLOR_BLACK 0X0000
#define ILI9341_COLOR_RED 0XF800
#define ILI9341_COLOR_GREEN 0X07E0
#define ILI9341_COLOR_GREEN2 0XB723
#define ILI9341_COLOR_BLUE 0X001F
#define ILI9341_COLOR_BLUE2 0X051D
#define ILI9341_COLOR_YELLOW 0XFFE0
#define ILI9341_COLOR_ORANGE 0XFBE4
#define ILI9341_COLOR_CYAN 0X07FF
#define ILI9341_COLOR_MAGENTA 0XA254
#define ILI9341_COLOR_GRAY 0X7BEF //1111 0111 1101 1110
#define ILI9341_COLOR_BROWN 0XBBCA

// COMMANDS
#define ILI9341_RESET 0X01
#define ILI9341_SLEEP_OUT 0X11
#define ILI9341_GAMMA 0X26
#define ILI9341_DISPLAY_OFF 0X28
#define ILI9341_DISPLAY_ON 0X29
#define ILI9341_COLUMN_ADDR 0X2A
#define ILI9341_PAGE_ADDR 0X2B
#define ILI9341_GRAM 0X2C
#define ILI9341_MAC 0X36
#define ILI9341_PIXEL_FORMAT 0X3A
#define ILI9341_WDB 0X51
#define ILI9341_WCD 0X53

```

```

#define ILI9341_RGB_INTERFACE 0XB0
#define ILI9341_FRC                                0XB1
#define ILI9341_BPC                                0XB5
#define ILI9341_DFC                                0XB6
#define ILI9341_POWER1                            0XC0
#define ILI9341_POWER2                            0XC1
#define ILI9341_VCOM1                             0XC5
#define ILI9341_VCOM2                             0XC7
#define ILI9341_POWERA                            0XCB
#define ILI9341_POWERB                            0XCF
#define ILI9341_PGAMMA                             0XE0
#define ILI9341_NGAMMA                             0XE1
#define ILI9341_DTCA                              0XE8
#define ILI9341_DTCB                              0XEA
#define ILI9341_POWER_SEQ                         0XED
#define ILI9341_3GAMMA_EN                         0XF2
#define ILI9341_INTERFACE                         0XF6
#define ILI9341_PRC                                0XF7

// SELECT ORIENTATION FOR LCD
typedef enum {
    LCD_ILI9341_ORIENTATION_PORTRAIT_1,
    LCD_ILI9341_ORIENTATION_PORTRAIT_2,
    LCD_ILI9341_ORIENTATION_LANDSCAPE_1,
    LCD_ILI9341_ORIENTATION_LANDSCAPE_2
} LCD_ILI9341_ORIENTATION_T;

// ORIENTATION, USED PRIVATE
typedef enum {
    LCD_ILI9341_LANDSCAPE,
    LCD_ILI9341_PORTRAIT
} LCD_ILI9341_ORIENTATION;

// LCD OPTIONS, USED PRIVATE
typedef struct {
    uint16_t width;
    uint16_t height;
    LCD_ILI9341_ORIENTATION orientation; // 1 = PORTRAIT; 0 = LANDSCAPE
} LCD_ILI931_OPTIONS_T;

// SELECT FONT
extern LCD_FONTDEF_T LCD_FONT_7X10;
extern LCD_FONTDEF_T LCD_FONT_11X18;
extern LCD_FONTDEF_T LCD_FONT_16X26;

/*
 * INITIALIZE ILI9341 LCD
 */
extern void LCD_ILI9341_INIT(void);

/*
 * INIT LCD PINS
 *
 * CALLED PRIVATE
 */
extern void LCD_ILI9341_INITLCD(void);

/*
 * SEND DATA TO LCD VIA SPI
 *
 * CALLED PRIVATE
 */

```

```

* PARAMETERS:
*   - UINT8_T DATA: DATA TO BE SENT
*/
EXTERN VOID LCD_ILI9341_SENDDATA(UINT8_T DATA);

/*
* SEND COMMAND TO LCD VIA SPI
*
* CALLED PRIVATE
*
* PARAMETERS:
*   - UINT8_T DATA: DATA TO BE SENT
*/
EXTERN VOID LCD_ILI9341_SENDCOMMAND(UINT8_T DATA);

/*
* SIMPLE DELAY
*
* PARAMETERS:
*   - VOLATILE UNSIGNED INT DELAY: CLOCK CYCLES
*/
EXTERN VOID LCD_ILI9341_DELAY(VOLATILE UNSIGNED INT DELAY);

/*
* SET CURSOR POSITION
*
* CALLED PRIVATE
*/
EXTERN VOID LCD_ILI9341_SETCURSORPOSITION(UINT16_T X1, UINT16_T Y1, UINT16_T X2, UINT16_T Y2);

/*
* DRAW SINGLE PIXEL TO LCD
*
* PARAMETERS:
*   - UINT16_T X: X POSITION FOR PIXEL
*   - UINT16_T Y: Y POSITION FOR PIXEL
*   - UINT16_T COLOR: COLOR OF PIXEL
*/
EXTERN VOID LCD_ILI9341_DRAWPIXEL(UINT16_T X, UINT16_T Y, UINT16_T COLOR);

/*
* FILL ENTIRE LCD WITH COLOR
*
* PARAMETERS:
*   - UINT16_T COLOR: COLOR TO BE USED IN FILL
*/
EXTERN VOID LCD_ILI9341_FILL(UINT16_T COLOR);

/*
* SHOW SELECTED QVGA IMAGE ON ENTIRE LCD
*
* PARAMETERS:
*   - UINT16_T IMAGE: ARRAY OF SELECTED IMAGE IN RB565 FORMAT
*/
EXTERN VOID LCD_ILI9341_DISPLAYIMAGE(UINT16_T IMAGE[ILI9341_PIXEL]);

/*
* ROTATE LCD
* SELECT ORIENTATION
*
* PARAMETERS:
*   - LCD_ILI9341_ORIENTATION_T ORIENTATION
*   - LCD_ILI9341_ORIENTATION_PORTRAIT_1: NO ROTATION

```

```

*           - LCD_ILI9341_ORIENTATION_PORTRAIT_2: ROTATE 180DEG
*           - LCD_ILI9341_ORIENTATION_LANDSCAPE_1: ROTATE 90DEG
*           - LCD_ILI9341_ORIENTATION_LANDSCAPE_2: ROTATE -90DEG
*/
EXTERN VOID LCD_ILI9341_ROTATE(LCD_ILI9341_ORIENTATION_T ORIENTATION);

/*
* PUT SINGLE CHARACTER TO LCD
*
* PARAMETERS:
*   - UINT16_T X: X POSITION OF TOP LEFT CORNER
*   - UINT16_T Y: Y POSITION OF TOP LEFT CORNER
*   - CHAR C: CHARACTER TO BE DISPLAYED
*   - LCD_FONTDEF_T *FONT: POINTER TO USED FONT
*   - UINT16_T FOREGROUND: COLOR FOR CHAR
*   - UINT16_T BACKGROUND: COLOR FOR CHAR BACKGROUND
*/
EXTERN VOID LCD_ILI9341_PUTC(UINT16_T X, UINT16_T Y, CHAR C, LCD_FONTDEF_T *FONT, UINT16_T FOREGROUND,
UINT16_T BACKGROUND);

/*
* PUT STRING TO LCD
*
* PARAMETERS:
*   - UINT16_T X: X POSITION OF TOP LEFT CORNER OF FIRST CHARACTER IN STRING
*   - UINT16_T Y: Y POSITION OF TOP LEFT CORNER OF FIRST CHARACTER IN STRING
*   - CHAR *STR: POINTER TO FIRST CHARACTER
*   - LCD_FONTDEF_T *FONT: POINTER TO USED FONT
*   - UINT16_T FOREGROUND: COLOR FOR STRING
*   - UINT16_T BACKGROUND: COLOR FOR STRING BACKGROUND
*/
EXTERN VOID LCD_ILI9341_PUTS(UINT16_T X, UINT16_T Y, CHAR *STR, LCD_FONTDEF_T *FONT, UINT16_T FOREGROUND,
UINT16_T BACKGROUND);

/*
* GET WIDTH AND HEIGHT OF BOX WITH TEXT
*
* PARAMETERS:
*   - CHAR *STR: POINTER TO FIRST CHARACTER
*   - LCD_FONTDEF_T *FONT: POINTER TO USED FONT
*   - UINT16_T *WIDTH: POINTER TO VARIABLE TO STORE WIDTH
*   - UINT16_T *HEIGHT: OINTER TO VARIABLE TO STORE HEIGHT
*/
EXTERN VOID LCD_ILI9341_GETSTRINGSIZE(CHAR *STR, LCD_FONTDEF_T *FONT, UINT16_T *WIDTH, UINT16_T *HEIGHT);

/*
* DRAW LINE TO LCD
*
* PARAMETERS:
*   - UINT16_T X0: X COORDINATE OF STARTING POINT
*   - UINT16_T Y0: Y COORDINATE OF STARTING POINT
*   - UINT16_T X1: X COORDINATE OF ENDING POINT
*   - UINT16_T Y1: Y COORDINATE OF ENDING POINT
*   - UINT16_T COLOR: LINE COLOR
*/
EXTERN VOID LCD_ILI9341_DRAWLINE(UINT16_T X0, UINT16_T Y0, UINT16_T X1, UINT16_T Y1, UINT16_T COLOR);

/*
* DRAW RECTANGLE ON LCD
*
* PARAMETERS:
*   - UINT16_T X0: X COORDINATE OF TOP LEFT POINT
*   - UINT16_T Y0: Y COORDINATE OF TOP LEFT POINT

```

```

* - UINT16_T X1: X COORDINATE OF BOTTOM RIGHT POINT
* - UINT16_T Y1: Y COORDINATE OF BOTTOM RIGHT POINT
* - UINT16_T COLOR: RECTANGLE COLOR
*/
EXTERN VOID LCD_ILI9341_DRAWRECTANGLE(UINT16_T X0, UINT16_T Y0, UINT16_T X1, UINT16_T Y1, UINT16_T COLOR);

/*
* DRAW FILLED RECTANGLE ON LCD
*
* PARAMETERS:
* - UINT16_T X0: X COORDINATE OF TOP LEFT POINT
* - UINT16_T Y0: Y COORDINATE OF TOP LEFT POINT
* - UINT16_T X1: X COORDINATE OF BOTTOM RIGHT POINT
* - UINT16_T Y1: Y COORDINATE OF BOTTOM RIGHT POINT
* - UINT16_T COLOR: RECTANGLE COLOR
*/
EXTERN VOID LCD_ILI9341_DRAWFILLEDRECTANGLE(UINT16_T X0, UINT16_T Y0, UINT16_T X1, UINT16_T Y1, UINT16_T
COLOR);

/*
* DRAW CIRCLE ON LCD
*
* PARAMETERS:
* - INT16_T X0: X COORDINATE OF CENTER CIRCLE POINT
* - INT16_T Y0: Y COORDINATE OF CENTER CIRCLE POINT
* - INT16_T R: CIRCLE RADIUS
* - UINT16_T COLOR: CIRCLE COLOR
*/
EXTERN VOID LCD_ILI9341_DRAWCIRCLE(INT16_T X0, INT16_T Y0, INT16_T R, UINT16_T COLOR);

/*
* DRAW FILLED ON LCD
*
* PARAMETERS:
* - INT16_T X0: X COORDINATE OF CENTER CIRCLE POINT
* - INT16_T Y0: Y COORDINATE OF CENTER CIRCLE POINT
* - INT16_T R: CIRCLE RADIUS
* - UINT16_T COLOR: CIRCLE COLOR
*/
EXTERN VOID LCD_ILI9341_DRAWFILLEDRCIRCLE(INT16_T X0, INT16_T Y0, INT16_T R, UINT16_T COLOR);

#endif

```

Література

1. STM32CubeMX

https://my.st.com/cas/login?service=https%3A%2F%2Fmy.st.com%2Fcontent%2Fmy_st_com%2Fen%2Fproducts%2Fdevelopment-tools%2Fsoftware-development-tools%2Fstm32-software-development-tools%2Fstm32-configurators-and-code-generators%2Fstm32cubemx.html

2. Reference manual. STM32F405xx/07xx, STM32F415xx/17xx, STM32F42xxx and STM32F43xxx advanced ARM®-based 32-bit MCUs.pdf

https://www.st.com/content/st_com/en/search.html#q=Reference%20manual.%20STM32F405xx/07xx-t=resources-page=1

3. STM32F429 Discovery

https://www.st.com/content/st_com/en/search.html#q=STM32F429%20Discovery-t=tools-page=1

4. ILI9341.pdf

5. OV7670_DS_(1_4).pdf

6. OV7670 SCH.pdf

7. Конспект лекцій з дисципліни «Мікропроцесорні системи»

НАВЧАЛЬНЕ ВИДАННЯ

ДОСЛІДЖЕННЯ ВУЗЛА ВВЕДЕННЯ ВІДЕОІНФОРМАЦІЇ В МПС

Методичні вказівки
до лабораторної роботи № 3
з дисципліни «Мікропроцесорні системи»
для студентів спеціальності 6.123.00.00 «Комп'ютерна інженерія»

Укладачі

Пуйда В.Я., к. т. н, доцент

Редактор

Комп'ютерне верстання

Здано у видавництво . Підписано до друку
Формат 70х100/16. Папір офсетний. Друк на різнографі
Умовн. друк. арк. Обл.-вид. арк..
Тираж прим. Зам..

Видавництво Національного університету "Львівська політехніка"
Реєстраційне свідоцтво ДК №751 від 27.12.2001 р.

Поліграфічний центр Видавництва
Національного університету "Львівська політехніка"

Вул.. Ф. Колесси, 2. Львів, 79000